

Schulungsunterlagen

Table of contents:

- MySQL/MariaDB für fortgeschrittene Entwickler
 - Zielgruppe & Voraussetzungen
 - Agenda
 - Modul 1: Advanced SQL & Transaction Management
 - Modul 2: Data Management & Migration
 - Modul 3: Programming, Tuning & PMM
 - Tech Stack & Setup
 - Szenario: Employees Database (Extended)
- Verbindung, Setup & CLI Tools
 - Verbindung zur Datenbank aufbauen
 - Der Login
 - Wichtige Flags für Entwickler
 - Die Tool-Suite
 - mysqladmin
 - mysqlshow
 - mysqldump & mysqlimport
 - Konfigurationsdateien (.my.cnf)
 - 3. Setup der Umgebung
 - Schritt 1: Docker Container starten
 - Schritt 2: Verbindung testen
- Advanced SQL: Joins, Union & Subqueries
 - JOIN-Typen im Detail
 - 1. INNER JOIN
 - 2. OUTER JOIN (LEFT / RIGHT)
 - 3. CROSS JOIN
 - 4. SELF JOIN
 - UNION und UNION ALL
 - Subqueries
 - IN, ANY, ALL, EXISTS
 - Scalar Subqueries
- Transaktionen & Locking
 - 1. Das Bank-Problem (ACID)
 - Syntax in MySQL
 - 2. Isolation Levels
 - 1. READ UNCOMMITTED (Dirty Read)

- 2. READ COMMITTED
- 3. REPEATABLE READ (MySQL Standard)
- 4. SERIALIZABLE
- 3. Locking & Deadlocks
 - Pessimistic Locking
 - Das Deadlock-Problem
- ACID & BASE-Prinzipien
 - ACID-Prinzipien von relationalen Datenbanken
 - BASE-Prinzipien von NoSQL-Datenbanken
- Views, Materialized Views & Virtual Columns
 - 1. Views (Sichten)
 - Warum Views nutzen?
 - 2. Materialized Views
 - 3. Generated Columns (Virtual Columns)
 - Typ A: VIRTUAL (Default)
 - Typ B: STORED
 - Use Case: JSON Indexing
 - Mini-Übung
- Lab: Grundlagen
 - Vorbereitung
 - Aufgabe 1: Der Import (Big Data)
 - Aufgabe 2: Der "Big Join" mit Bun
 - Aufgabe 3: JSON & Virtual Columns
- JSON & Window Functions
 - 1. JSON Support
 - Zugriff auf Daten
 - Suchen in JSON
 - Manipulation
 - MariaDB vs. MySQL
 - MariaDB Spezifische Funktionen
 - 2. Window Functions
 - Das Prinzip
 - Beispiel 1: Ranking
 - Beispiel 2: Running Total (Laufende Summe)
 - Beispiel 3: Lead & Lag (Zeitreisen)
 - Mini-Übung
- GIS: Geodaten in MySQL

- 1. Datentypen
- 2. Das Koordinatensystem (SRID)
- 3. Praxis-Beispiel: Die Pizza-Suche
 - Tabelle anlegen
 - Daten einfügen
 - Umkreissuche (Radius)
- 4. Polygon-Suche (Liefergebiet)
- 5. Theorie & Vergleich
 - Wann reicht MySQL GIS?
 - Wann brauchst du PostGIS?
 - Und MariaDB?
 - Quellen
- Fulltext Search
 - Fulltext Search
 - Index erstellen
 - Modus 1: Natural Language (Google-Style)
 - Modus 2: Boolean Mode
- Inkompatibilitäten & Feature-Unterschiede
 - 1. Grundsätzliche Positionierung
 - 2. Zusätzliche Storage Engines (nur in MariaDB)
 - 3. Erweiterte MariaDB-Funktionen (fehlen in MySQL)
 - 4. Wichtige funktionale Inkompatibilitäten
 - JSON-Handling
 - Rollen und Berechtigungen
 - Performance Schema
 - Standardbinaries
 - 5. Technische Unterschiede im Verhalten
 - 6. Authentifizierung und Replikation
 - 7. Nicht unterstützte MySQL-Funktionen in MariaDB
 - 8. Unterschiede bei Systemvariablen und Metadaten
 - 9. Praktische Auswirkungen
 - Migration und Kompatibilität
 - Einsatzszenarien
 - Zusammengefasst
- Lab: Advanced Features
 - Aufgabe 1: Der Deadlock Simulator
 - Aufgabe 2: JSON Produktkatalog

- Aufgabe 3: Fulltext Search Engine
- Stored Routines, Triggers & Events
 - 1. Stored Routines: Code in der DB
 - Pro & Contra
 - Stored Procedure
 - Aufruf
 - Deep Dive: Parameter (IN, OUT, INOUT)
 - 1. IN (Input)
 - 2. OUT (Output)
 - 3. INOUT (Input & Output)
 - Zusammenfassung
 - Stored Function
 - Warum diese Keywords?
 - Nutzung
 - Control Flow
 - 2. Triggers: Automatische Reaktionen
 - Use Case: Live Materialized View (Inkrementelles Update)
 - 3. Events: Der eingebaute Cronjob
- Tuning & Profiling
 - 1. EXPLAIN (Der Röntgenblick)
 - Die wichtigsten Spalten
 - 2. Profiling (Die Stoppuhr)
 - Software Profiling (App-Side)
 - 3. Schema Tuning Best Practices
 - Datentypen
 - Normalisierung vs. Denormalisierung
 - Indizes
- Software Profiling mit PMM
 - Architektur
 - Wie PMM Daten sammelt (Internals)
 - 1. Slow Query Log
 - 2. Performance Schema (perfschema)
 - Query Analytics (QAN)
 - Funktionsweise
 - Was tun, wenn es brennt?
- Lab: Die Meerwelt Suche
 - Vorbereitung

- Aufgabe 1: Die Stored Procedure
- Aufgabe 2: Der Bun Search Client
- Aufgabe 3: Testen
- Bonus
- Übungen: Joins, Unions & Subqueries
 - Aufgabe 1: Der einfache Join
 - Aufgabe 2: Manager-Historie
 - Aufgabe 3: Gehalts-Check (Aggregation)
 - Aufgabe 4: Missing Data (Left Join)
 - Aufgabe 5: The Big Union
 - Bonusübungen
- Übungen: Views & Virtual Columns
 - Aufgabe 1: Die "Current Employees" View
 - Aufgabe 2: Virtual Column für Namen
 - Aufgabe 3: Reporting View
 - Bonusübungen
- Übungen: Transaktionen & Locking
 - Aufgabe 1: Der sichere Geldtransfer
 - Aufgabe 2: Dirty Read Simulation
 - Aufgabe 3: Deadlock provozieren
 - Aufgabe 4: Pessimistic Locking
 - Bonusübungen
- Übungen: Funktionen, Window Functions & JSON
 - Aufgabe 1: Window Functions (Ranking)
 - Aufgabe 2: Window Functions (Lag)
 - Aufgabe 3: JSON Basics
 - Aufgabe 4: Eigene Funktion (Stored Function)
 - Bonusübungen
- Übungen: Stored Procedures, Triggers & Events
 - Aufgabe 1: Die "Hire" Prozedur
 - Aufgabe 2: Salary Protection Trigger
 - Aufgabe 3: Audit Event
 - Aufgabe 4: Cursor Loop
 - Bonusübungen
- Übungen: Tuning, Indexing & Profiling
 - Aufgabe 1: The Slow Query
 - Aufgabe 2: Composite Index

- Aufgabe 3: Sorting Optimization
- Aufgabe 4: App-Side Profiling (Bun)
- Bonusübungen
- Übung 07: GIS (Geodaten)
 - Szenario
 - Aufgaben
 - 1. Tabelle erstellen
 - 2. Daten einfügen
 - 3. Distanz berechnen
 - 4. Umkreissuche
 - 5. Bonus: Polygon
 - 6. Bonus: GeoJSON
- Der Umzug: Migration zu MariaDB
 - Szenario
 - Aufgabe 1: Der "perfekte" Export
 - ToDo
 - Aufgabe 2: Der Import
 - ToDo
 - Aufgabe 3: Best Practices & Diskussion
- Replikation in MySQL und MariaDB
 - Warum eigentlich Replikation?
 - Ziele der Replikation
 - Wie funktioniert Replikation in MySQL und MariaDB?
 - Ablauf im Detail
 - Eigenschaften
- Warum Galera Cluster?
 - Grenzen der Standard-Replikation
 - 1. Datenverlust bei Ausfall (Asynchronität)
 - 2. Komplexer Failover
 - 3. Keine Schreib-Skalierung
 - Die Lösung: Galera Cluster
 - Synchronität (Virtuell)
 - Multi-Master (Active-Active)
 - Automatische Node-Provisionierung
 - Vergleich: Standard vs. Galera
 - Wann Galera?
- Installation PMM

- MySQL Cheatsheet
 - 1. Grundlegende Abfragen (SELECT)
 - 2. Filtern (WHERE)
 - 3. Sortieren (ORDER BY)
 - 4. Aggregation & Gruppierung
 - Aggregat-Funktionen
 - GROUP BY
 - HAVING
 - 5. Daten Manipulation (DML)
 - 6. Nützliche Funktionen
 - String
 - Datum & Zeit
 - Logik
 - 7. Common Table Expressions (CTE)
 - 8. JSON Support
 - Tabelle erstellen
 - Daten einfügen
 - Daten abfragen (Extract)
 - Daten ändern (Update)
 - Nützliche JSON Funktionen
 - MariaDB Besonderheiten (JSON)
 - 9. Window Functions
 - Syntax
 - Ranking
 - Value Functions
- Rating
 - Vielen Dank für die Teilnahme am Seminar!

MySQL/MariaDB für fortgeschrittene Entwickler

Willkommen zum Experten-Kurs! Wir lassen die Grundlagen hinter uns und tauchen tief in die Architektur, Performance-Optimierung und fortgeschrittene Entwicklung mit **aktuellen LTS Versionen** von MySQL und MariaDB ein.

Dieser Kurs ist speziell für Entwickler konzipiert, die performante, skalierbare und sichere Datenbank-Anwendungen bauen müssen.

Zielgruppe & Voraussetzungen

Du solltest mit `SELECT`, `INSERT`, `UPDATE` und einfachen `JOIN`s bereits sicher umgehen können. Wir fokussieren uns auf:

- **Migration:** Strategien für den Wechsel von MySQL zu MariaDB.
- **Performance:** Query Tuning, Index-Strategien und Profiling (inkl. PMM).
- **Programmierung:** Business-Logik direkt in der Datenbank (Stored Procedures, Triggers).
- **Internals:** Transaktionen, Locks und Isolation Levels verstehen.

Agenda

Modul 1: Advanced SQL & Transaction Management

Wir starten mit dem Fundament für komplexe Anwendungen. Wie verhalten sich Daten unter Last und im Mehrbenutzerbetrieb?

- **Verbindung & Tools:** CLI Profi-Tipps (`mysql`, `mysqladmin`).

- **Complex Queries:** Deep Dive in `JOIN`-Typen, `UNION` Strategien und Subquery-Optimierung.
- **ACID & Transaktionen:** Was passiert bei `ROLLBACK` und `COMMIT` wirklich?
- **Locking:** Deadlocks verstehen und vermeiden (Row-Level Locking, Shared vs. Exclusive).

Modul 2: Data Management & Migration

Daten sind das Gold deines Unternehmens. Wir lernen, sie effizient zu bewegen und zu transformieren.

- **Import/Export:** Umgang mit großen Dumps (`mysqldump`, `mysqlimport`).
- **Migration Strategy: Special Focus:** Wir migrieren einen Datensatz live von MySQL zu MariaDB und prüfen die Kompatibilität.
- **Advanced Features:** Views, Materialized Views Workarounds, Virtual Columns.
- **Special Types:** JSON, GIS (Geodaten) und Volltextsuche.

Modul 3: Programming, Tuning & PMM

Der "Performance-Tag". Wir automatisieren Logik und finden jeden Flaschenhals.

- **Stored Programs:** Variablen, Stored Procedures, Functions und Error Handling (`SIGNAL`).
- **Automation:** Trigger und Events (der interne Cronjob).
- **Dynamic SQL:** Dynamische Code-Generierung.
- **Profiling & Tuning:** `EXPLAIN` verstehen, Schema-Optimierung.
- **Monitoring: Special Focus:** Setup und Nutzung von **PMM (Percona Monitoring and Management)** zur Analyse von Slow Queries.

Tech Stack & Setup

Wir nutzen eine moderne, containerisierte Umgebung.

- **MySQL LTS** (Container)
- **MariaDB LTS** (Container)

- **PMM Server 2.x** (Container)
- **Bun (TypeScript)** für Skripte und Client-Simulationen.

Szenario: Employees Database (Extended)

Wir arbeiten mit einem erweiterten Datensatz der "Employees" Datenbank (~300k Datensätze), um echte Performance-Effekte messbar zu machen.

Verbindung, Setup & CLI Tools

Vergiss GUIs für einen Moment. Echte Power-User und Skripte nutzen die CLI. Wir schauen uns die wichtigsten Tools an, die MySQL und MariaDB mitbringen.

Verbindung zur Datenbank aufbauen

Der Klassiker `mysql` ist mächtiger als man denkt.

Der Login

```
mysql -u root -p
```

 **SECURE PASSWORD**

Vermeide es, das Passwort direkt im Befehl mitzugeben (`-pSecret`). Es taucht sonst in der `history` auf. Nutze lieber Konfigurationsdateien oder den Prompt.

Wichtige Flags für Entwickler

- `--safe-updates`: Verhindert `UPDATE` oder `DELETE` ohne `WHERE` Klausel oder Limit.
- `--xml` / `--html`: Gibt Query-Ergebnisse direkt in Format X aus.
- `-e "QUERY"`: Führt ein Query aus und beendet sich (perfekt für Shell-Skripte).

```
# Beispiel: Quick Check ohne Shell zu öffnen
mysql -u user -pDBNAME -e "SELECT count(*) FROM employees;"
```

Die Tool-Suite

Neben dem Client gibt es Helfer für Administration und Daten.

mysqladmin

Ein Admin-Tool für Server-Operationen.

- Check, ob der Server lebt: `mysqladmin ping`
- Prozessliste anzeigen: `mysqladmin processlist`
- Status-Variablen: `mysqladmin extended-status`

mysqlshow

Schneller Überblick über Datenbanken, Tabellen und Spalten.

```
mysqlshow -u root -p employees
```

mysqldump & mysqlimport

(Dazu mehr in **Modul 2** beim Thema Migration & Backup).

Konfigurationsdateien (`.my.cnf`)

Statt Passwörter zu tippen, legst du sie sicher in deinem Home-Verzeichnis ab.

```
# ~/.my.cnf
[client]
user=myuser
password=mypassword
host=localhost
```

Rechte nicht vergessen: `chmod 600 ~/.my.cnf!`

3. Setup der Umgebung

Wir wollen keine Zeit mit Installationen verschwenden. Deshalb nutzen wir **Docker**.

Hier sind alle benötigten Dateien: [assets.zip](#)

Schritt 1: Docker Container starten

Es ist ein `docker-compose.yml` vorbereitet, das zwei Dienste liefert:

1. **MySQL 8.0**: Die Datenbank selbst.
2. **phpMyAdmin**: Eine Web-Oberfläche zur Verwaltung.

Wechsle in das `assets` Verzeichnis und starte die Container:

```
docker-compose up -d
```



TROUBLESHOOTING

Falls der Port 3306 belegt ist (z.B. durch eine lokale MySQL Installation), stoppe deinen lokalen Server oder ändere den Port in der `docker-compose.yml` auf `3307:3306`.

Schritt 2: Verbindung testen

1. **phpMyAdmin öffnen**: Gehe in deinem Browser auf <http://localhost:8080>.

2. **Einloggen**:

- **System**: MySQL
- **Server**: `mysql` (Das ist der Service-Name im Docker-Netzwerk)
- **Username**: `root`
- **Password**: `root`
- **Database**: `app_db`

3. **Bun (TypeScript) testen:** Wir nutzen Bun als Laufzeitumgebung für unsere Skripte.

```
# Dependencies installieren
bun install

# Test-Skript starten
bun run index.ts
```

Wenn du `Hello World` in der Konsole siehst, bist du bereit!

 **LINK ZUR EMPLOYEES-DATENBANK**

[Employees](#)

Advanced SQL: Joins, Union & Subqueries

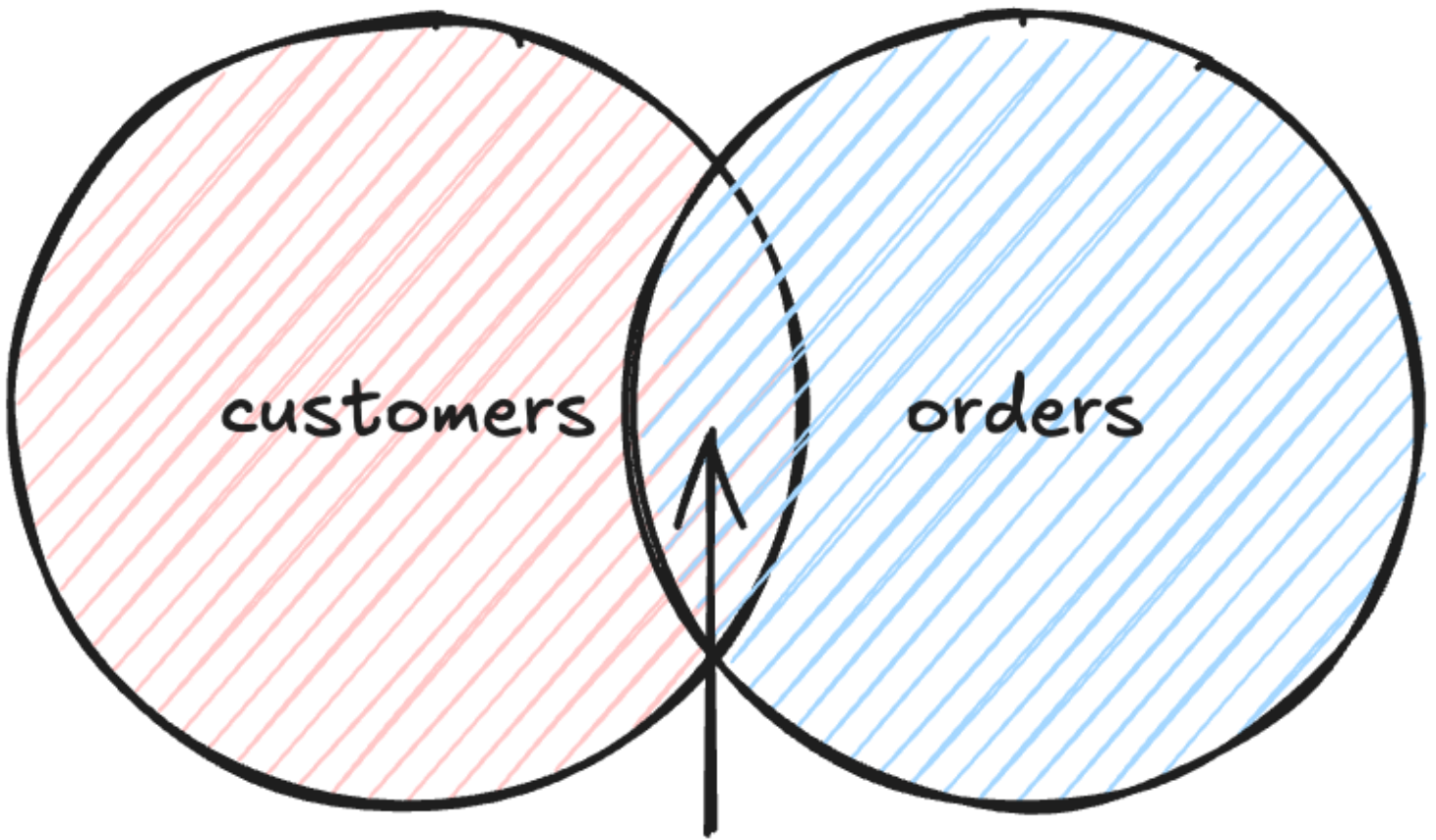
Wir bauen komplexe Berichte und Analysen. Hier trennt sich die Spreu vom Weizen: Wie effizient sind deine Joins?

JOIN-Typen im Detail

1. INNER JOIN

Der Standard. Nur Matches aus beiden Tabellen.

Inner Join



-> Nur Kunden MIT Bestellungen

```
SELECT e.first_name, d.dept_name
FROM employees e
INNER JOIN dept_emp de ON e.emp_no = de.emp_no
INNER JOIN departments d ON de.dept_no = d.dept_no;
```

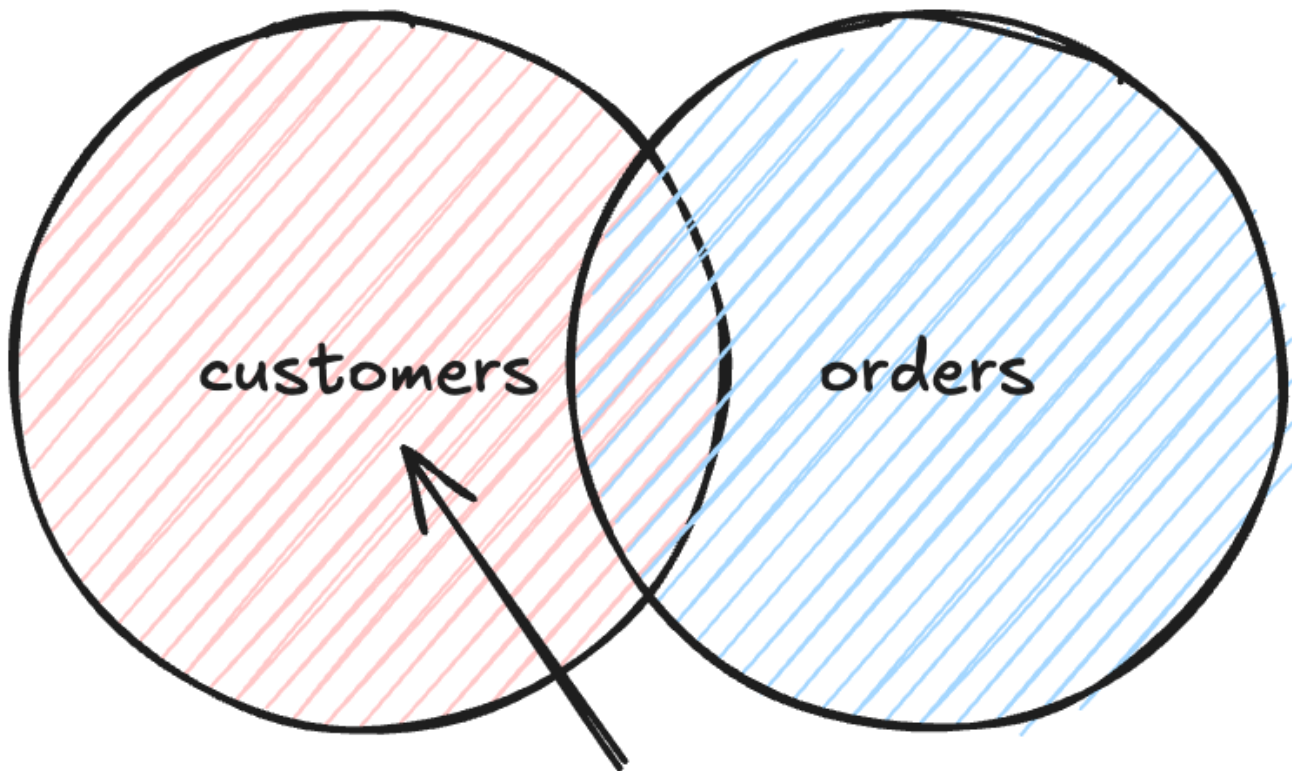
2. OUTER JOIN (LEFT / RIGHT)

Alles von einer Seite, Matches von der anderen. `NULL` wenn kein Match.

- **LEFT JOIN:** Alle Zeilen der *linken* Tabelle.

- **RIGHT JOIN:** Alle Zeilen der *rechten* Tabelle (selten genutzt, meist reicht LEFT JOIN durch Umstellen).

Left Join



- > Alle Kunden (auch ohne Bestellungen)
- > Bestellungen können "null" sein

! USE CASE

Finde alle Kunden, die *noch nie* bestellt haben: `SELECT c.name FROM customers c LEFT JOIN orders o ON c.id = o.c_id WHERE o.id IS NULL;`

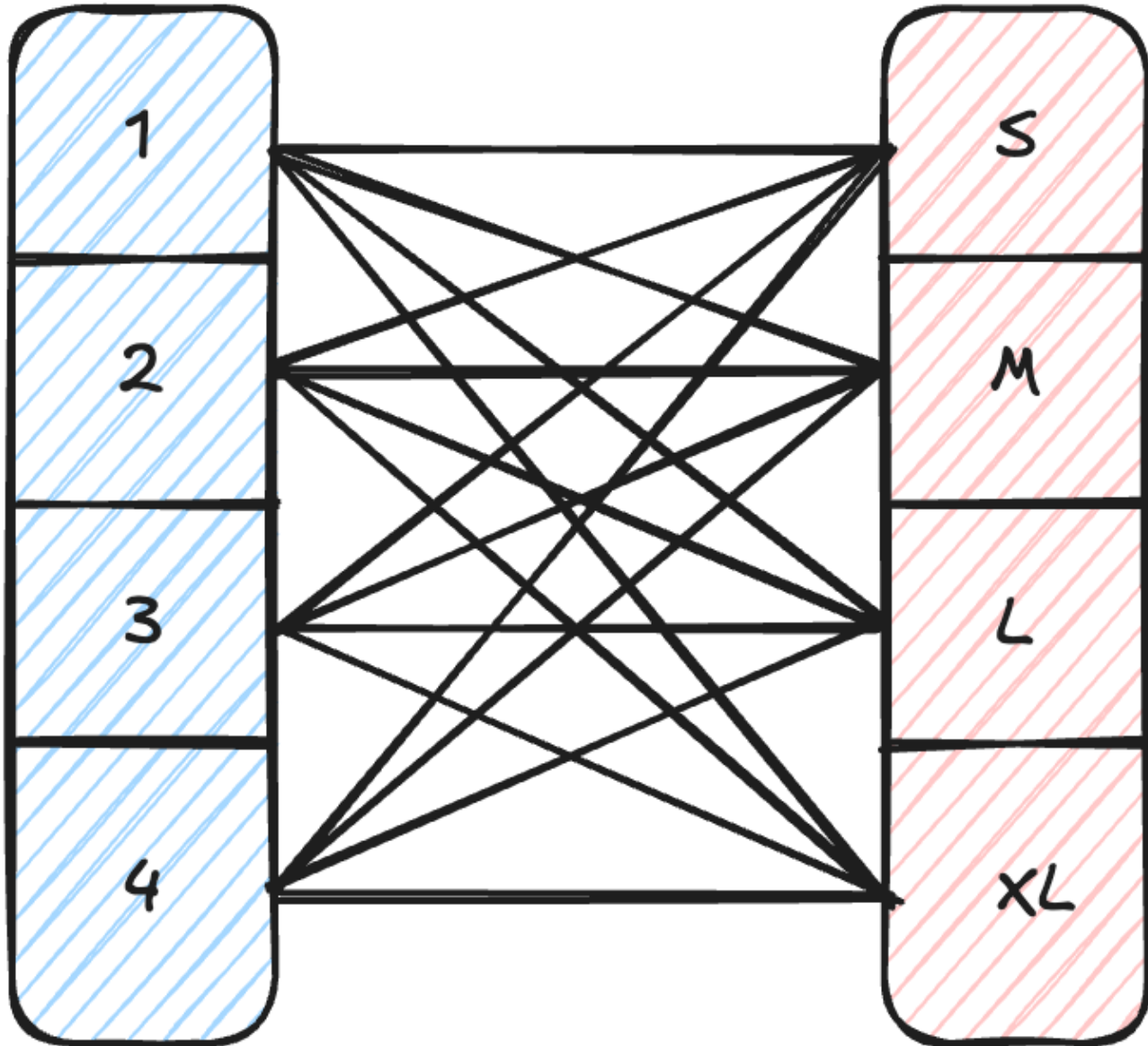
3. CROSS JOIN

Das kartesische Produkt. Jede Zeile mit jeder Zeile.

Cross Join

products

sizes



-> jedes Produkt wird mit jeder Größe kombiniert = 16 Kombinationen

P1S, P1M, P1L, P1XL

P2S, P2M, P2L, P2XL

etc...

- Vorsicht: 1000 Zeilen x 1000 Zeilen = 1.000.000 Ergebnisse.
- Nutzen: Generieren von Testdaten oder Kombinationstabellen (z.B. "Jeder Wochentag für jeden Mitarbeiter").

4. SELF JOIN

Eine Tabelle mit sich selbst verknüpfen.

- Nutzen: Hierarchien (Vorgesetzter ist auch Mitarbeiter) oder Vergleiche innerhalb einer Gruppe.

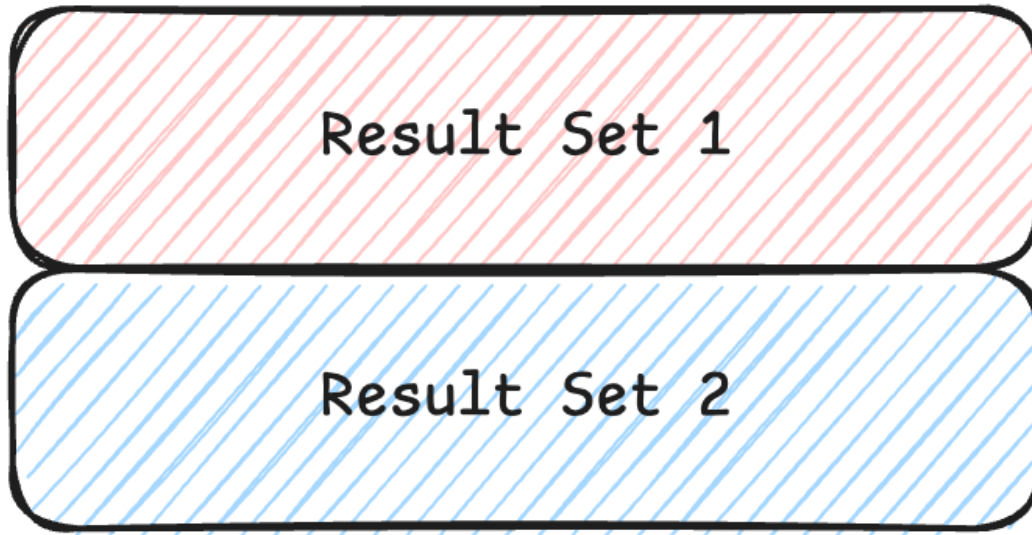
UNION und UNION ALL

Kombiniert Result-Sets von zwei Queries (müssen gleiche Spaltenanzahl/-typen haben).

- **UNION**: Entfernt Duplikate (Langsam, da implizites **DISTINCT**).

- `UNION ALL`: Behält Duplikate (Schnell).

Union



-> Result Sets werden untereinander angezeigt

-> `[UNION ALL]` Kann duplikate enthalten dafür schnell

-> `[UNION]` Entfernt Duplikate, muss dafür sortieren (CPU & Memory)

💡 PERFORMANCE

Nutze immer `UNION ALL`, es sei denn, du brauchst zwingend die Duplikat-Bereinigung. Der Performance-Unterschied bei großen Datenmengen ist extrem.

Subqueries

Abfragen in Abfragen.

IN, ANY, ALL, EXISTS

- `WHERE id IN (SELECT ...)`: Gut für kleine Listen.
- `WHERE EXISTS (SELECT ...)`: Oft performanter als `IN` bei großen Subqueries, da die DB beim ersten Match abbrechen kann ("Semi-Join").

Scalar Subqueries

Eine Subquery, die genau **einen Wert** (eine Zeile, eine Spalte) liefert. Kann überall stehen, wo ein Wert stehen kann (z.B. im `SELECT` Teil).

```
SELECT
    name,
    salary,
    (SELECT AVG(salary) FROM salaries) as avg_salary -- Scalar Subquery
FROM employees;
```

Achtung: Wird pro Zeile ausgeführt! Bei großen Listen Performance-Killer.
Besser: Window Functions (Tag 2) oder Join.

Transaktionen & Locking

Heute verlassen wir die Welt der einfachen Abfragen und kümmern uns um **Datenintegrität**. Wie wird sichergestellt, dass die Daten auch dann korrekt sind, wenn der Server abstürzt oder 1000 User gleichzeitig darauf zugreifen?

1. Das Bank-Problem (ACID)

Stell dir vor, es werden 100€ an einen Freund überwiesen. Das sind zwei Schritte in der Datenbank:

1. `UPDATE accounts SET balance = balance - 100 WHERE id = ME;`
2. `UPDATE accounts SET balance = balance + 100 WHERE id = FRIEND;`

Was passiert, wenn nach Schritt 1 der Strom ausfällt? Das Geld ist weg, aber beim Freund nicht angekommen. Katastrophe!

Hier kommen **Transaktionen** ins Spiel. Jede relationale Datenbank (wie MySQL mit InnoDB) verspricht **ACID**:

Syntax in MySQL

```
START TRANSACTION;
```

```
UPDATE accounts SET balance = balance - 100 WHERE id = 1;  
-- ... hier könnte alles mögliche passieren ...  
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
```

```
COMMIT; -- Erst jetzt wird es dauerhaft gespeichert  
-- Oder: ROLLBACK; -- Macht alles rückgängig
```

2. Isolation Levels

Das "I" in ACID ist das komplizierteste. Wie stark sollen parallele Transaktionen voneinander abgeschottet sein?

MySQL bietet 4 Level. Sie lassen sich pro Session setzen: `SET SESSION TRANSACTION ISOLATION LEVEL ...`

1. READ UNCOMMITTED (Dirty Read)

"Ich sehe alles, auch das, was noch nicht fertig ist."

- **Gefahr:** Man liest Daten von einer Transaktion, die später vielleicht `ROLLBACK` macht. Die Daten waren also nie real.
- **Nutzen:** Fast nie. Nur für grobe Statistiken ("Ungefähr wie viele Rows?").

2. READ COMMITTED

"Ich sehe nur das, was wirklich committet wurde."

- Das ist der Standard in PostgreSQL und Oracle.
- **Phänomen:** Wenn die gleiche SELECT-Query zweimal in einer Transaktion ausgeführt wird, könnten unterschiedliche Ergebnisse kommen (weil zwischendurch jemand anderes committet hat).

3. REPEATABLE READ (MySQL Standard)

"Ich lebe in meiner eigenen Zeitblase."

- Wenn eine Transaktion gestartet wird, macht MySQL quasi einen "Snapshot".
- Egal was andere machen (löschen, einfügen, ändern), man sieht immer den Stand vom Beginn der Transaktion.
- **Vorteil:** Konsistente Reports.

4. SERIALIZABLE

"Einer nach dem anderen."

- Extrem striktes Locking. Wenn gelesen wird, können andere nicht schreiben.

- **Nachteil:** Langsam und viele Timeouts.

3. Locking & Deadlocks

Um Isolation zu garantieren, muss MySQL Zeilen sperren (Locking).

Pessimistic Locking

Es wird davon ausgegangen, dass es Konflikte geben wird. Deshalb wird die Zeile gesperrt, **bevor** sie gelesen/bearbeitet wird.

```
START TRANSACTION;
-- "SELECT ... FOR UPDATE" sperrt die Zeilen exklusiv.
-- Niemand anderes kann sie ändern oder sperren, bis die Transaktion fertig
ist.
SELECT * FROM products WHERE id = 1 FOR UPDATE;

-- Jetzt kann man sicher sein, dass der Bestand sich nicht geändert hat.
UPDATE products SET stock = stock - 1 WHERE id = 1;
COMMIT;
```

Das Deadlock-Problem

Ein Deadlock ist eine Pattsituation.

- Transaktion A hält Lock auf Zeile 1 und will Zeile 2.
- Transaktion B hält Lock auf Zeile 2 und will Zeile 1.
- **Ergebnis:** Beide warten ewig.

MySQL erkennt das automatisch und bricht eine der beiden Transaktionen ab (Fehler 1213).

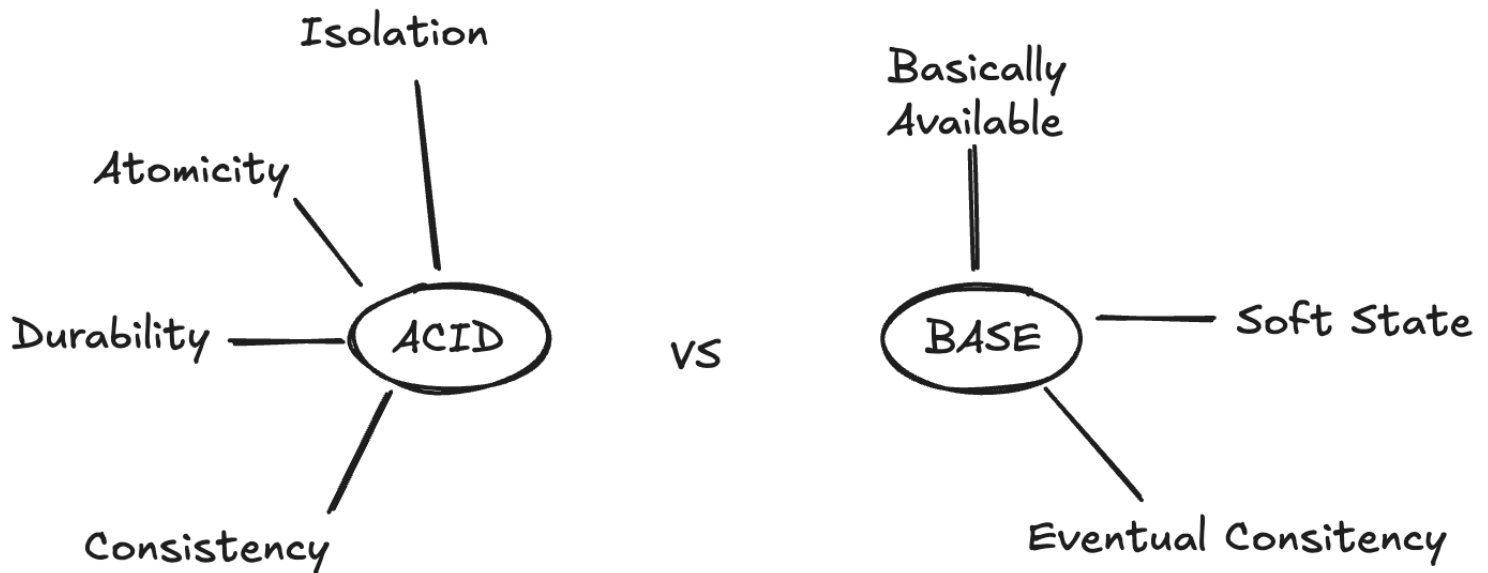


UMGANG MIT DEADLOCKS

Deadlocks sind in komplexen Apps normal.

1. **Reihenfolge:** Versuche, Tabellen/Zeilen immer in der gleichen Reihenfolge zu sperren.
2. **Retry:** Baue in der Applikation eine Logik ein, die bei Fehler 1213 die Transaktion einfach nochmal probiert.

ACID & BASE-Prinzipien



ACID-Prinzipien von relationalen Datenbanken

Die vier Eigenschaften **A** - **C** - **I** - **D** beschreiben, wie Datenbank-Transaktionen zuverlässig ablaufen sollen:

1. Atomicity (Atomarität)

- „Alles oder nichts“: Eine Transaktion wird entweder vollständig ausgeführt oder gar nicht.
- Beispiel: Beim Geldüberweisen muss das Abbuchen vom Konto A **und** das Gutschreiben auf Konto B passieren. Wenn ein Schritt fehlschlägt, wird die ganze Transaktion zurückgerollt.

2. Consistency (Konsistenz)

- Nach einer Transaktion muss die Datenbank immer in einem gültigen Zustand sein.
- Regeln, Integritätsbedingungen und Constraints dürfen nicht verletzt werden.
- Beispiel: Wenn eine Spalte nur positive Werte erlaubt, darf eine Transaktion keinen negativen Wert eintragen.

3. Isolation (Isolation)

- Gleichzeitige Transaktionen dürfen sich nicht gegenseitig beeinflussen.
- Für jede Transaktion sieht es so aus, als ob sie alleine ausgeführt würde.
- Beispiel: Zwei Personen kaufen gleichzeitig das letzte Konzertticket. Durch Isolation wird verhindert, dass beide es bekommen.

4. Durability (Dauerhaftigkeit)

- Einmal bestätigte Transaktionen bleiben dauerhaft gespeichert auch bei Stromausfall oder Systemabsturz.
- Umgesetzt wird das z. B. durch Logs, Backups oder Replikation.

BASE-Prinzipien von NoSQL-Datenbanken

BASE ist das Gegenstück zu ACID und beschreibt, wie viele verteilte, hochskalierbare Systeme arbeiten. Es steht für: **Basically Available – Soft state – Eventual consistency**

1. Basically Available (Grundsätzlich verfügbar)

- Das System garantiert, dass es **immer antwortet** auch wenn manche Teile fehlerhaft sind.
- Beispiel: Ein Onlineshop zeigt dir die Produktliste, auch wenn ein Server gerade ausgefallen ist.

2. Soft State (Weicher Zustand)

- Der Zustand der Daten kann sich **auch ohne neue Transaktion verändern**, weil Hintergrundprozesse (Replikation, Synchronisation) laufen.
- Beispiel: Zwei Datenbanken gleichen sich nachträglich ab – der aktuelle Zustand kann sich währenddessen noch ändern.

3. Eventual Consistency (Schlussendlich konsistent)

- Die Daten sind nicht sofort überall identisch, aber **mit der Zeit** gleichen sie sich an.

- Beispiel: Du likest ein Foto bei Instagram dein Freund sieht es vielleicht erst 2 Sekunden später, aber irgendwann ist der Zustand bei allen gleich.

Views, Materialized Views & Virtual Columns

Wie hält man die Datenbank sauber und performant, wenn die Anforderungen komplexer werden?

1. Views (Sichten)

Eine View ist im Grunde eine **gespeicherte Query**, die wie eine Tabelle aussieht.

Warum Views nutzen?

1. **Abstraktion:** Deine Applikation muss nicht wissen, dass die Daten aus 5 verschiedenen Tabellen gejoined werden. Sie macht einfach `SELECT * FROM v_user_details`.
2. **Sicherheit:** Du kannst einem User Zugriff auf eine View geben, aber den Zugriff auf die darunterliegenden Tabellen (z.B. mit Passwörtern) verweigern.
3. **Refactoring:** Wenn du dein Tabellen-Schema änderst, kannst du die View anpassen, ohne dass der App-Code geändert werden muss.

```
CREATE OR REPLACE VIEW v_current_salaries AS
SELECT e.emp_no, e.first_name, e.last_name, s.salary
FROM employees e
JOIN salaries s ON e.emp_no = s.emp_no
WHERE s.to_date > NOW(); -- Logik "Was ist aktuell?" zentral gekapselt
```

⚠ PERFORMANCE

In MySQL sind Views (meistens) **keine** Performance-Optimierung. Die Query wird jedes Mal zur Laufzeit ausgeführt (Merge Algorithm).

2. Materialized Views

In Datenbanken wie Oracle oder PostgreSQL gibt es "Materialized Views". Diese speichern das Ergebnis der Query physisch auf der Disk. Das ist extrem schnell beim Lesen, muss aber aktualisiert werden.

MySQL hat das **nicht eingebaut**. Aber es lässt sich simulieren!

Das Pattern:

1. Erstelle eine echte Tabelle für das Ergebnis (`mv_report`).
2. Fülle sie initial mit Daten.
3. Aktualisiere sie regelmäßig (z.B. jede Nacht via Event oder sofort via Trigger).

```
-- 1. Zieltable
CREATE TABLE mv_sales_report (
  category VARCHAR(50),
  total_amount DECIMAL(10,2),
  PRIMARY KEY (category)
);

-- 2. Refresh (z.B. jede Nacht)
TRUNCATE TABLE mv_sales_report;
INSERT INTO mv_sales_report
SELECT category, SUM(amount)
FROM sales
GROUP BY category;
```

3. Generated Columns (Virtual Columns)

Seit MySQL 5.7 gibt es `Generated Columns`

Stell dir vor, du hast eine Spalte, die sich automatisch aus anderen berechnet.

Typ A: VIRTUAL (Default)

Wird **on-the-fly** beim Lesen berechnet.

- Verbraucht keinen Speicherplatz auf der Disk.
- Kostet etwas CPU beim Lesen.
- Gut für einfache Berechnungen (`price * quantity`).

Typ B: STORED

Wird beim **Schreiben** (`INSERT`/`UPDATE`) berechnet und physisch gespeichert.

- Verbraucht Speicherplatz.
- Kostet Performance beim Schreiben.
- **Der Killer-Feature:** Du kannst einen **INDEX** darauf legen! Okay seit 8.0 geht das auch auf Virtual Columns.

Use Case: JSON Indexing

MySQL ist ein hervorragender JSON Store. Aber wie sucht man effizient in JSON?

```
CREATE TABLE users (  
  id INT PRIMARY KEY,  
  data JSON  
);  
  
-- Langsam (Full Scan)  
SELECT * FROM users WHERE data->>'$.email' = 'test@example.com';  
  
-- Die Lösung: Virtual Column + Index  
ALTER TABLE users  
ADD COLUMN email VARCHAR(255)  
GENERATED ALWAYS AS (data->>'$.email') STORED;  
  
CREATE INDEX idx_email ON users(email);  
  
-- Jetzt nutzt MySQL automatisch den Index!  
SELECT * FROM users WHERE email = 'test@example.com';
```

Mini-Übung

Erstelle eine Tabelle `rectangles` mit `width` und `height`. Füge eine Virtual Column `area` hinzu, die automatisch die Fläche berechnet. Füge ein paar Zeilen ein und prüfe, ob `area` korrekt gefüllt wird.

Lab: Grundlagen

Vorbereitung

Installiere BUN: `curl -fsSL https://bun.sh/install | bash`

1. Stelle sicher, dass deine Docker-Container laufen (`docker-compose up -d`).
2. Gehe in das `assets` Verzeichnis und installiere die Abhängigkeiten:

```
cd assets
bun install
```

Aufgabe 1: Der Import (Big Data)

Es werden Daten benötigt! Das "Employees" Dataset ist perfekt, um Performance zu testen.

1. **Download:** Lade das Repository `test_db` herunter (ZIP oder `git clone`).
2. **Import:** Importiere die `employees.sql` in deine Docker-Datenbank.
 - *Tipp:* Du kannst `mysql` lokal installieren oder den `mysql` Client im Docker-Container nutzen.
3. **Check:** Prüfe in phpMyAdmin, ob die Tabellen da sind. `employees` sollte ca. 300.000 Zeilen haben.

Aufgabe 2: Der "Big Join" mit Bun

Es soll eine komplexe Auswertung gefahren werden. Schreibe ein TypeScript-Skript `lab1_joins.ts`.

Die Frage: "Wer sind die **aktuellen** Manager jeder Abteilung, wie heißen sie, und was verdienen sie **aktuell**?"

Du brauchst:

- `departments` (Abteilungsname)
- `dept_manager` (Verknüpfung Manager -> Abteilung)
- `employees` (Namen)
- `salaries` (Gehalt)

Anforderungen:

1. Nutze `bun` für die Verbindung.
2. Achte darauf, nur `to_date > NOW()` Einträge zu nehmen (sonst hast du auch Ex-Manager).
3. Gib das Ergebnis schön formatiert in der Konsole aus (`console.table`).

```
// Skeleton
import { SQL } from "bun";

const db = new SQL("mysql://root:root@localhost:3306/employees");

async function main() {
  // Dein Code hier...
  // Beispiel: const result = await db`SELECT NOW()`;

  console.log("Ready to query!");
}
main();
```

Aufgabe 3: JSON & Virtual Columns

Es wird ein modernes Logging-System simuliert.

1. Erstelle eine Tabelle `app_logs` mit:
 - `id` (Auto Increment)
 - `created_at` (Timestamp)
 - `payload` (JSON)
2. Schreibe ein Skript, das **10.000 Log-Einträge** generiert und einfügt.

- Format: `{"level": "ERROR", "msg": "Something went wrong", "user_id": 123}`
 - Mische verschiedene Level (`INFO`, `WARN`, `ERROR`).
3. **Benchmark 1:** Suche alle `ERROR` Logs mit einer normalen JSON Query (`WHERE payload->'$.level' = 'ERROR'`). Miss die Zeit (nutze `console.time`).
4. **Optimierung:**
- Füge eine **Stored Generated Column** `log_level` hinzu.
 - Lege einen Index darauf.
5. **Benchmark 2:** Suche erneut (jetzt über die neue Spalte).



BUN PERFORMANCE

Bun ist sehr schnell. Um wirklich einen Unterschied zu messen, brauchst du vielleicht mehr als 10.000 Zeilen. Trau dich ruhig an 100.000!

JSON & Window Functions

MySQL 8.0 hat Features eingeführt, die die Grenze zwischen relationalen und NoSQL-Datenbanken verschwimmen lassen.

1. JSON Support

Früher wurden dynamische Daten oft als Text gespeichert oder EAV (Entity-Attribute-Value) Tabellen gebaut (Grusel!). Heute nutzt man den nativen JSON Datentyp.

MySQL speichert JSON in einem optimierten Binärformat (schneller Zugriff auf einzelne Keys, kein Parsen des ganzen Textes nötig).

Zugriff auf Daten

Man nutzt den Pfeil-Operator `->` (gibt JSON zurück) oder `->>` (gibt Text zurück, ohne Anführungszeichen).

```
SELECT
  id,
  data->>'$.name' as product_name,
  data->'$.tags[0]' as first_tag
FROM products;
```

Suchen in JSON

```
-- Finde alle Produkte, die den Tag "red" haben
SELECT * FROM products
WHERE JSON_CONTAINS(data->'$.tags', '"red"');

-- Finde alle Produkte, wo der Preis > 50 ist
SELECT * FROM products
WHERE data->'$.price' > 50;
```

Manipulation

JSON lässt sich atomar ändern, ohne dass der ganze String neu geschrieben werden muss.

```
UPDATE products
SET data = JSON_SET(data, '$.price', 99.99, '$.status', 'active')
WHERE id = 1;
```

❗ WANN JSON?

Nutze JSON für **Attribute**, die sich je nach Produkt unterscheiden (z.B. T-Shirt Größe vs. Festplatten-Kapazität). Nutze normale Spalten für **Metadaten**, nach denen oft gefiltert/sortiert wird (Preis, Erstellt am, Name).

MariaDB vs. MySQL

MariaDB geht beim Thema JSON **bewusst einen anderen Weg** als MySQL. Zwar existiert in MariaDB ein `JSON` Datentyp, technisch ist er jedoch **kein binärer JSON-Typ**, sondern ein Alias für `LONGTEXT`.

1. **Speicherung:** `JSON` ist intern `LONGTEXT` mit optionaler Validierung.
2. **Validierung:** Gültigkeit von JSON wird über `CHECK` Constraints oder Funktionen wie `JSON_VALID()` sichergestellt.

```
-- MariaDB Syntax
CREATE TABLE user_settings (
  user_id INT PRIMARY KEY,
  settings JSON CHECK (JSON_VALID(settings))
);
```

Vorteil: Schnelleres Schreiben, da kein binäres JSON-Parsing und keine interne Repräsentation beim Insert erzeugt wird. **Nachteil:** Langsameres Lesen bei komplexen Abfragen, da JSON bei jedem Zugriff zur Laufzeit geparkt wird.

Quelle: <https://mariadb.com/docs/server/reference/data-types/string-data-types/json>

MariaDB Spezifische Funktionen

MariaDB stellt JSON-Funktionen bereit, die sich **konzeptionell und semantisch** von MySQL unterscheiden. Besonders wichtig ist die Trennung zwischen skalaren und strukturellen Rückgaben.

- `JSON_VALUE(json, path)` Extrahiert **einen skalaren Wert** (String, Number, Boolean). Liefert **keinen JSON-Typ**, sondern direkt einen SQL-Skalar. Implizite Typkonvertierung möglich.
- `JSON_QUERY(json, path)` Extrahiert ein **JSON-Objekt oder Array** und gibt es als JSON Text zurück.

```
-- Gib mir den Preis als echte Zahl (DECIMAL), nicht als String
SELECT CAST(JSON_VALUE(data, '$.price') AS DECIMAL(10,2))
FROM products;
```

Dieses Verhalten ist bewusst näher an SQL Standard JSON Funktionen angelehnt als an MySQLs Implementierung.

Quelle: <https://mariadb.com/docs/server/reference/sql-functions/json-functions/>

2. Window Functions

Das wohl mächtigste Feature für Analysen. Window Functions erlauben Berechnungen über eine Gruppe von Zeilen, **ohne** sie mit `GROUP BY` zu einer einzigen Zeile zusammenzufassen.

Das Prinzip

```
FUNCTION() OVER (PARTITION BY ... ORDER BY ...)
```


Beispiel 1: Ranking

"Gib mir die Top 3 Gehälter pro Abteilung."

Mit `GROUP BY` unmöglich. Mit Window Functions einfach:

```
SELECT
    dept_no,
    emp_no,
    salary,
    RANK() OVER (PARTITION BY dept_no ORDER BY salary DESC) as ranking
FROM salaries;
```

Das Ergebnis behält alle Zeilen! Es gibt nur eine neue Spalte `ranking` (1, 2, 3, ...), die pro Abteilung neu bei 1 anfängt.

Beispiel 2: Running Total (Laufende Summe)

"Wie hat sich der Umsatz über den Monat entwickelt?"

```
SELECT
    date,
    amount,
    SUM(amount) OVER (ORDER BY date) as running_total
FROM sales;
```

Beispiel 3: Lead & Lag (Zeitreisen)

"Vergleiche den Umsatz heute mit dem von gestern."

```
SELECT
    date,
    revenue,
    LAG(revenue, 1) OVER (ORDER BY date) as yesterday_revenue,
    revenue - LAG(revenue, 1) OVER (ORDER BY date) as diff
FROM daily_sales;
```

Nutze die `salaries` Tabelle. Zeige für jeden Mitarbeiter das aktuelle Gehalt und das Gehalt davor (nutze `LAG` über `from_date`).

GIS: Geodaten in MySQL

Früher brauchte man PostGIS oder Elasticsearch für Geodaten. Heute kann MySQL das nativ und sehr effizient.

1. Datentypen

MySQL bietet spezielle Datentypen für geometrische Formen, die dem OGC (Open Geospatial Consortium) Standard folgen.

- **POINT**: Ein einzelner Punkt (z.B. Standort eines Users).
- **LINESTRING**: Eine Route oder Straße.
- **POLYGON**: Eine Fläche (z.B. Liefergebiet, Stadtgrenze).
- **GEOMETRY**: Kann jeden der obigen Typen speichern.

2. Das Koordinatensystem (SRID)

Die Erde ist keine Scheibe (meistens).

- **SRID 0 (Default)**: Eine unendliche flache Ebene. Distanzen sind einfach Pythagoras. Gut für CAD-Pläne oder Pixel-Koordinaten.
- **SRID 4326 (WGS 84)**: Das GPS-System. Längengrad und Breitengrad auf einer Kugel (Ellipsoid).

❗ WICHTIG

Wenn du GPS-Daten speicherst, nutze **immer** SRID 4326. Sonst sind deine Distanzberechnungen falsch!

3. Praxis-Beispiel: Die Pizza-Suche

Wir speichern Restaurants und suchen alles im Umkreis von 5km.

Tabelle anlegen

```
CREATE TABLE restaurants (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100),  
  -- POINT(Longitude Latitude) !  
  location POINT NOT NULL SRID 4326,  
  SPATIAL INDEX(location) -- Der Turbo für die Suche  
);
```

Daten einfügen

Achtung: Die Reihenfolge ist `POINT(Längengrad Breitengrad)` (X Y). In Google Maps ist es oft andersrum!

```
INSERT INTO restaurants (name, location)  
VALUES  
  ('Döner King', ST_GeomFromText('POINT(13.4000 52.5200)', 4326)),  
  ('Sushi Bar', ST_GeomFromText('POINT(13.4050 52.5200)', 4326));
```

Umkreissuche (Radius)

Wir nutzen `ST_Distance_Sphere`, um die Distanz in Metern auf der Erdkugel zu berechnen.

```
-- Finde alles im Umkreis von 1000 Metern um den Alexanderplatz  
SELECT name,  
       ST_Distance_Sphere(location, ST_GeomFromText('POINT(13.41 52.52)',  
4326)) as dist_m  
FROM restaurants  
WHERE ST_Distance_Sphere(location, ST_GeomFromText('POINT(13.41 52.52)',  
4326)) < 1000;
```

ST

ST steht für Spatial. Spatial ist ein Synonym für Geometrische Operationen. GIS steht üblicherweise für Geographic Information System.

Seit MySQL 8.0 gibt es Spatial Operationen.

4. Polygon-Suche (Liefergebiet)

Liegt ein Punkt in einem bestimmten Bereich?

```
SELECT name
FROM restaurants
WHERE ST_Within(location, ST_GeomFromText('POLYGON(...)', 4326));
```

5. Theorie & Vergleich

Wann reicht MySQL GIS?

Für **99% aller Web-Anwendungen** ist MySQL 8.0 völlig ausreichend.

- **Use Cases:** Store Locator, Umkreissuche, "Liefern wir zu dir?" (Point in Polygon), einfache Routen-Distanz.
- **Vorteil:** Du brauchst keinen separaten GIS-Server. Du hast ACID-Transaktionen, Backups und Replikation zusammen mit deinen relationalen Daten.

Wann brauchst du PostGIS?

PostgreSQL mit der PostGIS-Extension ist der Gold-Standard im GIS-Bereich. Ein Wechsel lohnt sich, wenn:

- Du **Rasterdaten** (Satellitenbilder, Höhenmodelle) verarbeiten musst.
- Du komplexe **Topologie-Prüfungen** brauchst (z.B. Netzwerkanalysen für Stromleitungen).
- Du Koordinatensysteme (Projektionen) direkt in der DB transformieren musst (`ST_Transform`).

Und MariaDB?

MariaDB unterstützt ebenfalls GIS (OGC Standard), aber es gibt technische Unterschiede:

- **Engine:** MySQL nutzt die **Boost.Geometry** Library. Seit Version 8.0 ist die Implementierung extrem strikt:
 - **SRID-Matching:** Operationen zwischen unterschiedlichen SRIDs werfen Fehler.
 - **Achsen-Reihenfolge:** MySQL 8.0 hält sich strikt an den EPSG-Standard (bei SRID 4326 ist das *Latitude, Longitude*). MariaDB und ältere MySQL-Versionen sind da oft laxer (X, Y).
- **Kompatibilität:** Grundlegende Funktionen (`ST_Contains`, `ST_Within`) funktionieren meist identisch. Bei komplexen Berechnungen auf der Erdkugel (Ellipsoid) ist MySQL 8.0 oft genauer, da es standardmäßig geodätische Berechnungen auf dem Ellipsoid durchführt, während MariaDB historisch eher planare Annäherungen nutzte (wobei neuere Versionen aufholen).

Quellen

- [Geography Support in MySQL 8.0](#) - Erklärung zur Ellipsoid-Unterstützung (SRID 4326).
- [Boost.Geometry Library](#) - Die Engine unter der Haube von MySQL 8.0.

Fulltext Search

Fulltext Search

`LIKE '%wort%'` ist langsam und dumm (findet "Wörter" nicht, wenn man nach "Wort" sucht). Fulltext Search ist die eingebaute Suchmaschine.

Index erstellen

Ein Fulltext Index funktioniert wie ein Buch-Index. Er speichert, welches Wort in welcher Zeile vorkommt.

```
ALTER TABLE articles ADD FULLTEXT(title, body);
```

Modus 1: Natural Language (Google-Style)

Versucht, die "besten" Treffer zu finden. Sortiert automatisch nach Relevanz.

```
SELECT *, MATCH(title, body) AGAINST('Database Tutorial') as score
FROM articles
WHERE MATCH(title, body) AGAINST('Database Tutorial');
```

Modus 2: Boolean Mode

Erlaubt Operatoren für präzise Kontrolle.

- `+`: Wort **muss** vorkommen.
- `-`: Wort **darf nicht** vorkommen.
- `*`: Wildcard (am Wortende).
- `" "`: Exakte Phrase.

```
-- Finde Artikel, die "Database" enthalten, aber NICHT "Oracle".
-- "MySQL" ist optional, erhöht aber den Score.
```

```
SELECT * FROM articles  
WHERE MATCH(title, body) AGAINST('+Database -Oracle MySQL' IN BOOLEAN MODE);
```

STOPWORDS

MySQL ignoriert sehr häufige Wörter ("the", "is", "and") automatisch. Das kann man konfigurieren, ist aber meistens gewollt.

Inkompatibilitäten & Feature-Unterschiede

Hier ist eine strukturierte und sachliche Zusammenfassung der Unterschiede und Inkompatibilitäten zwischen MariaDB Rolling und MySQL 8.0 (basierend auf der offiziellen MariaDB-Dokumentation).

Quelle

1. Grundsätzliche Positionierung

MariaDB versucht weiterhin hohe Kompatibilität zu MySQL zu halten, trotzdem existieren wichtige Unterschiede auf Funktionsebene und im Verhalten der Systeme.

2. Zusätzliche Storage Engines (nur in MariaDB)

MariaDB Rolling bietet zusätzliche Storage Engines, die MySQL nicht unterstützt:

- **ColumnStore:** Für verteilte Datenverarbeitung und Analytics.
- **MyRocks:** Eine Engine mit hoher Kompression, optimiert für Flash-Storage.
- **S3 Storage Engine:** Ermöglicht das Archivieren von Tabellen in z. B. Amazon S3.
- **Weitere:** CONNECT, SEQUENCE, Spider, FederatedX, OQGRAPH, SphinxSE u. a.

3. Erweiterte MariaDB-Funktionen (fehlen in MySQL)

MariaDB enthält mehrere Erweiterungen, die in MySQL fehlen:

- **Relationale Vektorfunktionen**
- **Galera-Cluster-Integration:** Synchroner Multi-Master Cluster.
- **Zeitbasierte Tabellen:** System-Versioned, Application-Time Period und Bitemporal Tables ("Zeitreisen" in SQL).
- **Flashback-DML-Rollback:** Rückgängigmachen von Änderungen ohne Backup-Restore.
- **Oracle-Kompatibilitätsmodi:** Erleichtert die Migration von Oracle DBs (PL/SQL Support).
- **Sequences:** Echte Sequenz-Objekte (`CREATE SEQUENCE`).
- **RETURNING-Klauseln:** `INSERT ... RETURNING id`.
- **Dynamische Spalten:** NoSQL-artiges Schema.
- **Oracle-ähnliche Features:** z. B. `EXECUTE IMMEDIATE`.
- **Erweiterte JSON-Funktionen**
- **Mehr Kollationen und Zeichenunterstützung**
- **Fortschritte beim Optimizer**
- **Zusätzliche Daten- und Index-Typen:** wie `UUID`, `INET4/6`.
- **Progress-Reporting:** für Operationen wie `ALTER TABLE`.
- **Mehr Kommandos:** z. B. erweiterte `FLUSH`-Funktionen.

4. Wichtige funktionale Inkompatibilitäten

JSON-Handling

- **Speicherung:** MariaDB speichert JSON intern als Text (`LONGTEXT`), nicht als binäres JSON wie MySQL.
- **Operatoren:** Bestimmte JSON-Operatoren (`->`, `->>`) aus MySQL werden teils unterschiedlich oder gar nicht unterstützt (kontextabhängig).
- **Vergleich:** Vergleichsverhalten von JSON unterscheidet sich.
- **Validierung:** Bei falschen JSON-Eingaben gibt MariaDB oft eine Warnung und `NULL` zurück, MySQL strikt einen Fehler.

Rollen und Berechtigungen

- **Auth:** MariaDB unterstützt keine Authentifizierung via Rollen (Mapping von User auf Rolle beim Login), MySQL schon.
- **Schema:** Unterschiedliche Darstellung von Rollen in `INFORMATION_SCHEMA`.
- **Tabellen:** Zusätzliche Rollen-Tabellen in MySQL fehlen in MariaDB.

Performance Schema

- MySQL aktiviert `performance_schema` standardmäßig.
- MariaDB hat diese standardmäßig deaktiviert (kann aber aktiviert werden).

AKTIVIEREN

per Config

```
[mysqld]  
performance_schema = ON
```

Standardbinaries

- MariaDB verwendet andere Binärdateinamen (`mariadb` statt `mysqld`).

5. Technische Unterschiede im Verhalten

- **Befehle:** MariaDB Rolling implementiert bestimmte Befehle (z. B. `FLUSH TABLES`) anders.
- **Abbruch:** In MariaDB kann jede Query abgebrochen werden, in MySQL nur `SELECT`.
- **Parameter:** Unterschiedlicher Umgang mit Optionen, z. B. kürzere vs vollständige Parametererkennung.

6. Authentifizierung und Replikation

- **GTID:** Nicht kompatible GTID-Implementierungen zwischen MariaDB und MySQL. Replikation ist oft schwierig.
- **Passwörter:** Nutzer mit dem MySQL-SHA256-Passwortalgorithmus (`caching_sha2_password`) funktionieren nicht in MariaDB Rolling.

7. Nicht unterstützte MySQL-Funktionen in MariaDB

- Lateral Derived Tables
- CIDR-Notation für Accounts
- MySQL X Plugin (Document Store API)
- `CREATE TABLESPACE` für InnoDB
- `SELECT FOR SHARE` (MySQL Syntax) ist in MariaDB Rolling so nicht verfügbar (nutze `LOCK IN SHARE MODE`).
- Ngram und MeCab Full-Text-Parser Plugins fehlen.

8. Unterschiede bei Systemvariablen und Metadaten

- Unterschiede im `INFORMATION_SCHEMA`.
- Unterschiede bei unterstützten Zeichensätzen und Kollationen.
- Einige Syntax-Unterschiede bei DDL-Operationen (z. B. `SHOW CREATE TABLE`).

9. Praktische Auswirkungen

Migration und Kompatibilität

Wo MariaDB als Drop-in-Ersatz fungieren kann, bleibt bestehen, aber bei neueren MySQL-Funktionen oder speziellen Feature-Sets kann es zu Inkompatibilitäten kommen (z. B. JSON, Rollen, Replikation).

Einsatzszenarien

- **MariaDB** eignet sich dort, wo erweiterte Engines und Funktionen benötigt werden.
 - **MySQL** wird typischerweise dort bevorzugt, wo Oracle-Enterprise-Features oder bestimmte Standard-Funktionen kritisch sind.
-

Zusammengefasst

MariaDB Rolling und MySQL 8.0 sind eng verwandt, aber nicht identisch. MariaDB hat zusätzliche Storage Engines und Funktionen, während MySQL einige Standardfeatures und Enterprise-Tools bietet, die MariaDB nicht oder anders implementiert. Insbesondere bei JSON-Handling, Rollenmanagement, Authentifizierung, Replikationsmodellen und bestimmten SQL-Funktionen bestehen klare Unterschiede, die bei Migrationen und im Betrieb berücksichtigt werden müssen.

Lab: Advanced Features

Heute wird es knifflig. Es werden echte Probleme (Deadlocks) simuliert und moderne Features (JSON Store) gebaut.

Aufgabe 1: Der Deadlock Simulator

Ein Deadlock passiert selten zufällig beim Entwickeln. Er muss erzwungen werden, um zu lernen, wie man ihn behandelt.

Ziel: Schreibe ein Bun-Skript `lab2_deadlock.ts`, das zwei parallele Transaktionen startet, die sich gegenseitig blockieren.

Ablauf:

1. Erstelle eine Tabelle `accounts` mit `id` und `balance`. Setze zwei Accounts (ID 1 und 2) ein.
2. Nutze `Promise.all`, um zwei asynchrone Funktionen gleichzeitig zu starten (`tx1` und `tx2`).
3. **Tx1:**
 - `START TRANSACTION`
 - `UPDATE accounts ... WHERE id = 1` (Lockt ID 1)
 - `await new Promise(r => setTimeout(r, 2000))` (Warte 2 Sekunden, damit Tx2 Zeit hat zu starten)
 - `UPDATE accounts ... WHERE id = 2` (Will Lock für ID 2 -> Wartet auf Tx2)
4. **Tx2:**
 - `START TRANSACTION`
 - `UPDATE accounts ... WHERE id = 2` (Lockt ID 2)
 - `await new Promise(r => setTimeout(r, 2000))`
 - `UPDATE accounts ... WHERE id = 1` (Will Lock für ID 1 -> Wartet auf Tx1)
5. **Ergebnis:** Fange den Fehler ab und gib aus, wer gewonnen hat.

Aufgabe 2: JSON Produktkatalog

Es wird ein flexibler Shop gebaut.

1. Erstelle Tabelle `products_json` (`id`, `name`, `attributes` JSON).
 2. Füge via Bun 5 Produkte ein. Nutze unterschiedliche Attribute!
 - Laptop: `{"ram": "16GB", "os": "macOS", "ports": ["usb-c", "hdmi"]}`
 - T-Shirt: `{"size": "L", "color": "blue", "material": "cotton"}`
 3. **Query 1:** Finde alle Produkte, die das Attribut `os` haben (egal welches).
 4. **Query 2:** Finde alle Produkte, die im Array `ports` den Wert `usb-c` haben (`JSON_CONTAINS`).
 5. **Update:** Ändere bei allen T-Shirts das Material auf "bio-cotton" (nutze `JSON_SET` oder `JSON_REPLACE`).
-

Aufgabe 3: Fulltext Search Engine

1. Nutze die `employees` Datenbank.
2. Es soll in den `titles` gesucht werden.
3. Erstelle einen `FULLTEXT` Index auf der Spalte `title`.
4. Finde alle Mitarbeiter, die "Engineer" im Titel haben, aber **nicht** "Senior" oder "Assistant".
 - *Tipp: Boolean Mode nutzen!*
5. Vergleiche die Performance (Laufzeit) mit einer normalen `LIKE '%Engineer%'` Suche.

Stored Routines, Triggers & Events

Heute wird Logik vom Applikations-Code (TypeScript) direkt in die Datenbank verlagert. Das ist ein mächtiges Werkzeug, aber auch ein zweiseitiges Schwert.

1. Stored Routines: Code in der DB

Pro & Contra

Pro	Contra
Performance: Weniger Daten müssen über das Netzwerk.	Debugging: Schwer zu debuggen (kein <code>console.log</code>).
Sicherheit: User darf Prozedur ausführen, aber Tabellen nicht lesen.	Version Control: Code liegt in der DB, nicht im Git (außer man nutzt Migrations).
Konsistenz: Logik ist für alle Clients (Web, App, Admin) gleich.	Vendor Lock-in: Schwerer zu PostgreSQL o.ä. zu wechseln.

Stored Procedure

Eine Prozedur wird mit `CALL` aufgerufen. Sie kann Daten ändern und Ergebnisse zurückliefern.

```
DELIMITER // -- Das Semikolon muss als Trennzeichen temporär geändert werden
```

```
CREATE PROCEDURE give_raise(  
  IN p_dept_no CHAR(4),  
  IN p_percent DECIMAL(5,2)
```



```

)
BEGIN
    -- Logik kapseln
    UPDATE salaries s
    JOIN dept_emp de ON s.emp_no = de.emp_no
    SET s.salary = s.salary * (1 + p_percent / 100)
    WHERE de.dept_no = p_dept_no
        AND s.to_date > NOW(); -- Nur aktuelle!

    -- Feedback geben
    SELECT CONCAT('Updated salaries for department ', p_dept_no) as msg;
END //

DELIMITER ;

```

Aufruf

```
CALL give_raise('d005', 10.00);
```

Deep Dive: Parameter (IN, OUT, INOUT)

MySQL kennt drei Arten von Parametern. Das ist wichtig, um zu verstehen, wie Daten fließen.

1. IN (Input)

- **Bedeutung:** Wert geht *rein*.
- **Verhalten:** Die Prozedur kann ihn lesen und intern ändern, aber die Änderung bleibt *in* der Prozedur.
- **Default:** Wenn nichts angegeben ist, ist es `IN`.
- **Use Case:** Filter, IDs, Rechenwerte.

In unserem Beispiel `give_raise` nutzen wir nur `IN`, weil wir Daten verarbeiten, aber nichts zurückgeben müssen (außer einer Message via SELECT).

2. OUT (Output)

- **Bedeutung:** Wert geht *raus*.

- **Verhalten:** Der Caller übergibt eine Variable (ohne Wert), die Prozedur füllt sie.
- **Use Case:** Berechnete Ergebnisse zurückgeben (statt SELECT).

Beispiel:

```
CREATE PROCEDURE count_employees(IN p_dept CHAR(4), OUT p_count INT)
BEGIN
    SELECT COUNT(*) INTO p_count FROM dept_emp WHERE dept_no = p_dept;
END;
```

Aufruf:

```
CALL count_employees('d005', @anzahl);
SELECT @anzahl;
```

3. INOUT (Input & Output)

- **Bedeutung:** Wert geht *rein*, wird geändert und geht *raus*.
- **Use Case:** Zähler, Akkumulatoren, Modifikation eines Wertes.

Beispiel:

```
CREATE PROCEDURE adjust_score(INOUT p_score INT)
BEGIN
    SET p_score = p_score + 10;
END;
```

Aufruf:

```
SET @score = 50;
CALL adjust_score(@score);
SELECT @score; -- 60
```

Zusammenfassung

Typ	Richtung	Caller sieht Änderung?	Typischer Zweck
IN	Rein	Nein	Filter, IDs (Standard)
OUT	Raus	Ja	Ergebnisse zurückgeben
INOUT	Rein & Raus	Ja	Werte modifizieren

Stored Function

Eine Funktion wird **in** einem SQL-Statement genutzt (wie `UPPER()` oder `NOW()`). Sie **muss** einen Wert zurückgeben.

```
CREATE FUNCTION get_manager_name(p_dept_no CHAR(4))
RETURNS VARCHAR(100)
DETERMINISTIC READS SQL DATA
BEGIN
    DECLARE v_name VARCHAR(100);

    SELECT CONCAT(e.first_name, ' ', e.last_name) INTO v_name
    FROM employees e
    JOIN dept_manager dm ON e.emp_no = dm.emp_no
    WHERE dm.dept_no = p_dept_no AND dm.to_date > NOW()
    LIMIT 1;

    RETURN v_name;
END;
```

Warum diese Keywords?

MySQL zwingt dich bei Stored Functions dazu, zu deklarieren:

1. Wie vorhersehbar ist das Ergebnis?

- `DETERMINISTIC`: Gleicher Input = Gleicher Output (z.B. `1+1`). MySQL kann das Ergebnis cachen.
- `NOT DETERMINISTIC`: Ergebnis ändert sich (z.B. `NOW()`, `RAND()`).

2. Wie greift die Funktion auf Daten zu?

- **NO SQL**: Nutzt kein SQL.
- **READS SQL DATA**: Liest nur (**SELECT**).
- **MODIFIES SQL DATA**: Schreibt (**INSERT**, **UPDATE**).
- **CONTAINS SQL**: Nutzt SQL, liest/schreibt aber keine Daten.

Nutzung

```
SELECT get_manager_name('d005');
```

Control Flow

Logik braucht Kontrollstrukturen. MySQL bietet die Klassiker:

Compound Statements:

- **IF ... THEN ... ELSE ... END IF;**
- **CASE ... WHEN ... THEN ... END CASE;**
- **WHILE ... DO ... END WHILE;**

2. Triggers: Automatische Reaktionen

Trigger feuern automatisch bei **INSERT**, **UPDATE** oder **DELETE**.

Use Cases:

- **Audit Logging**: Wer hat was geändert?
- **Validierung**: Daten prüfen, bevor sie gespeichert werden.
- **Berechnung**: Virtual Columns aktualisieren (bei komplexer Logik).

```
CREATE TRIGGER before_employee_insert
BEFORE INSERT ON employees
FOR EACH ROW
BEGIN
    -- Validierung: Niemand darf in der Zukunft eingestellt werden
```

```
IF NEW.hire_date > NOW() THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Hire date cannot be in the future!';
END IF;
END;
```

Use Case: Live Materialized View (Inkrementelles Update)

Statt `SUM()` jedes Mal neu zu berechnen, halten wir eine Statistik-Tabelle aktuell.

```
CREATE TRIGGER update_sales_after_order
AFTER INSERT ON orders
FOR EACH ROW
BEGIN
    -- Den User-Umsatz sofort erhöhen
    UPDATE user_stats
    SET total_spent = total_spent + NEW.amount
    WHERE user_id = NEW.user_id;
END;
```

VORSICHT

Trigger sind "unsichtbar". Ein Entwickler wundert sich vielleicht, warum sein INSERT fehlschlägt oder warum plötzlich Daten in einer anderen Tabelle auftauchen. Dokumentiere Trigger gut!

3. Events: Der eingebaute Cronjob

MySQL hat einen eigenen Scheduler. Perfekt für Aufräumarbeiten.

```
-- Muss aktiviert sein!
SET GLOBAL event_scheduler = ON;

CREATE EVENT clean_logs
```

```
ON SCHEDULE EVERY 1 DAY
STARTS '2024-01-01 03:00:00'
DO
    DELETE FROM logs WHERE created_at < NOW() - INTERVAL 30 DAY;

-- Beispiel 2: Pseudo Materialized View (Teure Reports nachts bauen)
CREATE EVENT refresh_daily_report
ON SCHEDULE EVERY 1 DAY STARTS '2025-01-01 03:00:00'
DO
BEGIN
    TRUNCATE TABLE sales_report_mv;

    INSERT INTO sales_report_mv
    SELECT category_id, SUM(amount)
    FROM orders
    GROUP BY category_id;
END;
```

Tuning & Profiling

"Die Datenbank ist langsam." - Der Satz, den jeder Entwickler fürchtet. Heute lernen wir, wie wir das beheben.

1. EXPLAIN (Der Röntgenblick)

Bevor wild Indizes erstellt werden, frage MySQL, wie es die Query ausführen will. Setze einfach `EXPLAIN` vor die Query.

```
EXPLAIN SELECT * FROM employees WHERE last_name = 'Facello';
```

Die wichtigsten Spalten

Spalte	Wert	Bedeutung	Gut/Schlecht
type	ALL	Full Table Scan. MySQL liest JEDE Zeile.	 Sehr Schlecht (außer bei winzigen Tabellen)
	index	Full Index Scan. Liest den ganzen Index.	 Naja
	range	Nutzt Index für einen Bereich (>, BETWEEN).	 Gut
	ref	Nutzt Index für exakten Match.	 Sehr Gut
	const	Primary Key Lookup (max 1 Treffer).	 Perfekt
key	NULL	Kein Index genutzt.	 Schlecht

Spalte	Wert	Bedeutung	Gut/Schlecht
rows	1000	MySQL <i>schätzt</i> , dass es 1000 Zeilen prüfen muss.	Je weniger, desto besser
Extra	Using filesort	Muss Ergebnisse extra sortieren (CPU intensiv).	● Vermeiden wenn möglich
	Using temporary	Erstellt temporäre Tabelle (RAM/Disk).	● Vermeiden

2. Profiling (Die Stoppuhr)

EXPLAIN zeigt den Plan. PROFILING zeigt die Realität.

```
-- 1. Aktivieren
SET profiling = 1;

-- 2. Query ausführen
SELECT ...;

-- 3. Report ansehen
SHOW PROFILES;
-- Query_ID | Duration | Query
-- 1        | 0.500    | SELECT ...

-- 4. Details ansehen
SHOW PROFILE FOR QUERY 1;
```

Hier sieht man genau:

- Sending data: Zeit für Netzwerk-Transfer.
- Creating sort index: Zeit für CPU-Sortierung.
- Waiting for table lock: Zeit für Warten auf andere.

Software Profiling (App-Side)

Oft liegt das Problem gar nicht in der Datenbank.

- **N+1 Problem:** Die Datenbank ist schnell (0.1ms), wird aber 10.000 Mal aufgerufen.
- **Large Payloads:** Die Query dauert 10ms, aber das Übertragen von 50MB JSON dauert 2 Sekunden.

Lösung: Messe auch im Applikations-Code!

```
console.time("db_query");  
const users = await db`SELECT * FROM users`;  
console.timeEnd("db_query"); // -> db_query: 12.4ms
```

3. Schema Tuning Best Practices

Datentypen

Nutze immer den kleinstmöglichen Typ.

- `TINYINT` (1 Byte) statt `INT` (4 Byte) für Status-Flags (0/1).
- `VARCHAR(20)` statt `VARCHAR(255)` wenn bekannt ist, dass es kurz ist (hilft bei Temp Tables im RAM).

Normalisierung vs. Denormalisierung

- **3. Normalform:** Keine Redundanz. Gut für Schreib-Performance (nur 1 Stelle ändern).
- **Denormalisierung:** Redundanz erlauben (z.B. `customer_name` auch in `orders` Tabelle speichern).
 - **Vorteil:** Spart JOINS beim Lesen -> Schneller.
 - **Nachteil:** Bei Namensänderung muss man an 2 Stellen updaten.



FAUSTREGEL

Starte normalisiert. Denormalisiere nur gezielt, wenn es Performance-Probleme gibt.

Indizes

- **Composite Indexes:** Ein Index über `(last_name, first_name)`.
 - Hilft bei `WHERE last_name = 'X'`.
 - Hilft bei `WHERE last_name = 'X' AND first_name = 'Y'`.
 - Hilft **NICHT** bei `WHERE first_name = 'Y'` (Links-Präfix-Regel)!

Software Profiling mit PMM

Percona Monitoring and Management (PMM) ist der Gold-Standard für Open-Source Datenbank-Monitoring.

Architektur

PMM folgt einer klassischen **Client-Server-Architektur**:

1. **PMM Server**: Die Kommandozentrale.
 - **Grafana**: Zur Visualisierung (Dashboards).
 - **VictoriaMetrics**: Speichert Time-Series-Daten (CPU, Disk I/O, MySQL Status).
 - **ClickHouse**: Speichert Query-Daten für QAN (Query Analytics).
 2. **PMM Client**: Ein Agent (`pmm-agent`), der auf deinem Datenbank-Server läuft.
 - Er managt Exporter wie `mysqld_exporter` (für Metriken) und `node_exporter` (für System-Daten).
 - Er schickt die Daten verschlüsselt an den Server.
-

Wie PMM Daten sammelt (Internals)

Woher weiß PMM, welche Query langsam ist? Es gibt zwei Hauptquellen.

1. Slow Query Log

Der klassische Weg. MySQL schreibt alle Queries, die länger als `long_query_time` dauern, in eine Datei.

- **Vorteil**: Funktioniert immer und überall.
- **Nachteil**: Teuer (Disk I/O), ungenau (nur "Zeit > X"), muss geparkt werden.

2. Performance Schema (`perfschema`)

Der moderne Weg. MySQL instrumentiert sich selbst und hält Metriken im RAM (`performance_schema.*` Tabellen).

- **Vorteil:** Extrem detailliert (Lock-Zeiten, Rows examined, Index usage), schnell (Memory), granular.
- **PMM:** Nutzt standardmäßig `perfschema`, um Daten für QAN zu sammeln (`--query-source=perfschema`).

❗ WARUM PERFORMANCE SCHEMA?

Das Performance Schema erlaubt Einblicke, die Logs nicht bieten: "Wartete diese Query auf einen Mutex?" oder "Hat sie einen Temp-Table auf Disk angelegt?".

Query Analytics (QAN)

Das QAN ist das Herzstück für Entwickler. Es beantwortet die Frage: "Warum ist die DB langsam?"

Funktionsweise

PMM sammelt Rohdaten (z.B. 1000 Ausführungen von `SELECT * FROM users WHERE id = ?`) und aggregiert sie.

1. **Fingerprinting:** PMM entfernt konkrete Werte.

- `SELECT * FROM users WHERE id = 5`
- `SELECT * FROM users WHERE id = 9`
- -> Werden zusammengefasst zu `SELECT * FROM users WHERE id = ?`.

2. **Aggregation:** PMM zeigt dir:

- **Count:** Wie oft lief die Query? (z.B. 10.000 Mal/Sek)
- **Load:** Wie viel Zeit hat sie insgesamt auf dem Server "verbrannt"?

Was tun, wenn es brennt?

1. QAN öffnen und Zeitraum des Spikes markieren.

2. Nach **Load** sortieren (nicht nach Einzel-Dauer!). Eine Query, die 10ms dauert, aber 1000x pro Sekunde läuft, ist oft schlimmer als eine, die 1x pro Stunde 5s dauert.
3. Query anklicken -> **EXPLAIN** Tab prüfen.
4. Optimieren (Index hinzufügen, Caching, Rewrite).

Lab: Die Meerwelt Suche

Wir bauen eine CLI-Suchmaschine für eine Meeresbiologie-Datenbank. Ziel: Eine performante Volltextsuche mit Bun und MySQL.

Vorbereitung

1. Erstelle eine neue Datei zB `search.ts`

2. **Daten importieren:**

```
-- Tabelle erstellen
CREATE TABLE IF NOT EXISTS ocean_articles (
  id INT AUTO_INCREMENT PRIMARY KEY,
  title VARCHAR(255),
  content TEXT,
  FULLTEXT(title, content)
);

-- Daten einfügen
INSERT INTO ocean_articles (title, content) VALUES
('Der Blauwal', 'Der Blauwal (Balaenoptera musculus) ist das größte bekannte Tier der Erdgeschichte. Er kann bis zu 33 Meter lang werden und 200 Tonnen wiegen. Seine Zunge wiegt so viel wie ein Elefant. Er ernährt sich fast ausschließlich von Krill.'),
('Korallenriffe', 'Korallenriffe sind komplexe Ökosysteme unter Wasser, die von riffbildenden Steinkorallen gebildet werden. Sie werden oft als "Regenwälder der Meere" bezeichnet, da sie eine enorme Artenvielfalt beherbergen. Das Great Barrier Reef ist das größte Riffsystem der Erde.'),
('Der Große Weiße Hai', 'Der Weiße Hai (Carcharodon carcharias) ist der größte Raubfisch der Welt. Er ist bekannt für seinen ausgeprägten Geruchssinn und seine Rolle als Spitzenprädatoren in den Ozeanen. Entgegen der landläufigen Meinung jagt er Menschen nicht gezielt.'),
('Tiefsee-Anglerfisch', 'In der absoluten Dunkelheit der Tiefsee lebt der Anglerfisch. Weibchen besitzen eine biolumineszente "Angel" am Kopf, um Beute anzulocken. Männchen sind oft winzig und verschmelzen parasitär mit dem Weibchen.'),
('Ozeanströmungen', 'Meeresströmungen wie der Golfstrom transportieren gewaltige Mengen Wasser und Wärme um den Globus. Sie beeinflussen das Weltklima
```

maßgeblich. Der Golfstrom sorgt beispielsweise für das milde Klima in Europa.'),
('Plastikverschmutzung', 'Millionen Tonnen Plastikmüll landen jährlich in den Ozeanen. Mikroplastik wurde bereits in den tiefsten Meeresgräben und im arktischen Eis gefunden. Es bedroht Meeresbewohner, die es mit Nahrung verwechseln.'),
('Die Intelligenz der Oktopusse', 'Kraken und Oktopusse gelten als die intelligentesten Wirbellosen. Sie können Werkzeuge nutzen, Gläser öffnen und komplexe Probleme lösen. Sie besitzen drei Herzen und blaues Blut.'),
('Biolumineszenz', 'Viele Meeresbewohner, besonders in der Tiefsee, können eigenes Licht erzeugen. Dies dient der Kommunikation, der Tarnung (Gegenbeleuchtung) oder dem Anlocken von Beute. Bekannte Beispiele sind Leuchtqualen und Laternenfische.'),
('Hydrothermale Quellen', 'Schwarze Raucher sind hydrothermale Quellen am Meeresboden. Hier tritt heißes, mineralreiches Wasser aus. Sie bilden die Lebensgrundlage für spezialisierte Ökosysteme, die ohne Sonnenlicht existieren (Chemosynthese).'),
('Der Marianengraben', 'Der Marianengraben ist die tiefste Stelle der Weltmeere, mit einer Tiefe von etwa 11.000 Metern. Der Druck dort unten ist mehr als 1000-mal höher als an der Oberfläche. Nur wenige spezialisierte Lebewesen können dort überleben.'),
('Seegraswiesen', 'Seegraswiesen sind wichtige Kohlenstoffspeicher (Blue Carbon) und bieten Lebensraum für viele Jungfische. Sie stabilisieren den Meeresboden und verbessern die Wasserqualität.'),
('Wale und Echoortung', 'Zahnwale wie Delfine und Pottwale nutzen Echoortung (Sonar), um sich zu orientieren und Beute zu finden. Sie senden Klicklaute aus und interpretieren das Echo, um ein "Hörbild" ihrer Umgebung zu erstellen.'),
('Gezeiten (Ebbe und Flut)', 'Die Gezeiten werden hauptsächlich durch die Anziehungskraft des Mondes und der Sonne verursacht. Der Tidenhub variiert weltweit stark, von wenigen Zentimetern bis zu über 10 Metern in der Bay of Fundy.'),
('El Niño', 'El Niño ist ein Klimaphänomen im pazifischen Ozean, bei dem sich das Oberflächenwasser ungewöhnlich stark erwärmt. Dies hat weltweite Auswirkungen auf Wetter, Fischerei und Landwirtschaft.'),
('Mangrovenwälder', 'Mangroven sind salztolerante Bäume, die in den Gezeitenzonen tropischer Küsten wachsen. Ihre Wurzeln bieten Schutz für Fische und Krebstiere und schützen die Küste vor Erosion und Sturmfluten.');

Aufgabe 1: Die Stored Procedure

Erstelle eine Stored Procedure `search_ocean`, die:

1. Einen Suchbegriff (`IN p_term VARCHAR(255)`) entgegennimmt.
2. Im `BOOLEAN MODE` in `title` und `content` sucht.
3. Die Ergebnisse nach Relevanz sortiert zurückgibt.
4. Maximal 5 Treffer liefert.

Aufgabe 2: Der Bun Search Client

Schreibe ein Skript `search.ts`, das via CLI Argument einen Suchbegriff annimmt und die Ergebnisse schön formatiert ausgibt.

Anforderungen:

- Verbindung zur DB herstellen.
- CLI Argument prüfen (wenn leer -> Fehler).
- Stored Procedure aufrufen.
- Ergebnisse anzeigen (Titel fett/farbig, Content kurz).

Code-Gerüst (`search.ts`):

```
import { SQL } from "bun";

// TODO: Passwort anpassen!
const db = new SQL("mysql://root:root@<DEINE IP>:3306/app_db");

const term = process.argv[2];

if (!term) {
  console.error("Bitte Suchbegriff angeben!");
  console.log("Usage: bun run search.ts <begriff>");
  process.exit(1);
}

console.log(`Suche in der Meerwelt nach: "${term}"...\n`);

try {

  // Hier dein Code

} catch (err) {
```



```
    console.error("Fehler:", err);  
  }  
  
  await db.close();
```

Aufgabe 3: Testen

Finde heraus:

1. Was frisst der Blauwal? (`bun run search.ts "Blauwal"`)
2. Wo leben Anglerfische? (`bun run search.ts "Tiefsee"`)
3. Suche nach "Fisch" aber ohne "Hai". (`bun run search.ts "+Fisch -Hai"`)

Bonus

- **Highlighting:** Kannst du den Suchbegriff im ausgegebenen Text farbig markieren? (Tipp: `text.replace(...)` und ANSI Codes).
- **Logging:** Erweitere die Stored Procedure, sodass jede Suche in einer Tabelle `search_history` (term, timestamp) gespeichert wird.

Übungen: Joins, Unions & Subqueries

Hier sind Aufgaben, um das Wissen zu vertiefen. Alle Aufgaben basieren auf der `employees` Datenbank.

Aufgabe 1: Der einfache Join

Ziel: Liste alle Mitarbeiter auf, die aktuell in der Abteilung "Sales" arbeiten.

- **Benötigte Tabellen:** `employees`, `dept_emp`, `departments`
- **Bedingung:** `to_date > NOW()` (nur aktuelle Verknüpfungen)
- **Ausgabe:** `first_name`, `last_name`, `dept_name`

Aufgabe 2: Manager-Historie

Ziel: Wer war alles Manager der Abteilung "Development"?

- **Benötigte Tabellen:** `dept_manager`, `departments`, `employees`
- **Sortierung:** Nach `from_date` aufsteigend.
- **Ausgabe:** `first_name`, `last_name`, `from_date`, `to_date`

Aufgabe 3: Gehalts-Check (Aggregation)

Ziel: Wie hoch ist das **durchschnittliche** Gehalt in jeder Abteilung (aktuell)?

- **Benötigte Tabellen:** `salaries`, `dept_emp`, `departments`
- **Hinweis:** Vergiss nicht, nur aktuelle Gehälter und Abteilungszugehörigkeiten zu filtern!

- **Ausgabe:** `dept_name`, `avg_salary` (gerundet auf 2 Stellen)

Aufgabe 4: Missing Data (Left Join)

Ziel: (Theoretisch) Finde alle Mitarbeiter, die **keinen** Titel haben.

- **Hinweis:** In der `employees` DB hat eigentlich jeder einen Titel. Um das zu testen, könnte man einen Dummy-User ohne Titel einfügen.
- **Query:** Schreibe eine Query mit `LEFT JOIN` zwischen `employees` und `titles`, die nur Mitarbeiter ohne Titel-Eintrag liefert.

Aufgabe 5: The Big Union

Ziel: Erstelle eine Liste aller "Manager" und aller "Senior Engineers".

- **Spalten:** `first_name`, `last_name`, `role` (Setze den Wert manuell auf 'Manager' oder 'Senior Dev')
- **Hinweis:** Nutze `UNION ALL` für bessere Performance, wenn Du weißt, dass es keine Überschneidungen gibt (oder `UNION` wenn doch).

Bonusübungen

1. **Self Join:** Finde alle Paare von Mitarbeitern, die den gleichen Nachnamen haben (gib nur die ersten 10 Paare aus).
2. **Cross Join:** Generiere eine (theoretische) Liste aller möglichen Kombinationen aus Abteilungen und Titeln.
3. **Subquery in FROM:** Finde alle Abteilungen, die mehr als 50.000 Einträge in der `dept_emp` Tabelle haben (nutze dazu eine Subquery in der FROM-Klausel oder eine CTE).
4. **EXISTS vs. IN:** Finde alle Manager, die auch noch eine normale Rolle als "Engineer" hatten. Löse die Aufgabe einmal mit `IN` und einmal mit `EXISTS`. Vergleiche (theoretisch) die Performance.
5. **Recursive CTE:** (Fortgeschritten) Erstelle eine rekursive CTE, die die Zahlen 1 bis 10 generiert. (Falls keine Hierarchie-Daten da sind, ist das

eine gute Übung für die Syntax).

6. **Natural Join:** Nutze `NATURAL JOIN` um `employees` und `dept_emp` zu verbinden. Was passiert? (Vorsicht: Es verbindet alle gleichnamigen Spalten!).
7. **Finding Gaps:** Angenommen, wir haben eine Tabelle mit fortlaufenden Nummern (IDs). Schreibe eine Query (z.B. mit Self-Join), die Lücken in der Nummerierung findet.
8. **Anti-Join:** Finde alle Abteilungen, die aktuell *keinen* Manager haben (falls das datentechnisch möglich wäre). Nutze dazu `LEFT JOIN ... WHERE ... IS NULL`.
9. **Aggregation mit Filter:** Berechne das Durchschnittsgehalt pro Abteilung, aber berücksichtige nur Gehälter über 60.000. Nutze dazu eine gefilterte Aggregation (oder `WHERE` vor `GROUP BY`).
10. **Complex Grouping:** Gruppieren die Mitarbeiter nach Einstellungsjahr (`YEAR(hire_date)`) und Geschlecht. Zähle die Anzahl pro Gruppe.

Übungen: Views & Virtual Columns

Aufgabe 1: Die "Current Employees" View

Die Tabellen `dept_emp` und `salaries` enthalten Historien. Das nervt bei täglichen Abfragen.

Ziel: Erstelle eine View `v_current_employees`.

- Sie soll **nur** aktuelle Daten enthalten (`to_date > NOW()`).
- **Spalten:** `emp_no`, `first_name`, `last_name`, `current_salary`, `current_dept_name`.

```
SELECT * FROM v_current_employees WHERE last_name = 'Facello';
```

Aufgabe 2: Virtual Column für Namen

Es wird oft nach dem vollen Namen gesucht oder dieser angezeigt.

Ziel:

1. Füge der Tabelle `employees` eine Generated Column `full_name` hinzu.
2. Definition: `CONCAT(first_name, ' ', last_name)`.
3. Entscheide: `VIRTUAL` oder `STORED`? (Tipp: Wenn danach gesucht werden soll, ist `STORED` + Index besser).

Aufgabe 3: Reporting View

Ziel: Erstelle eine View `v_dept_stats`, die pro Abteilung folgende Metriken liefert:

- Anzahl der Mitarbeiter
- Durchschnittsgehalt
- Summe aller Gehälter

Frage: Ist diese View performant? Was passiert, wenn man `SELECT * FROM v_dept_stats` macht? (Antwort: MySQL berechnet die Aggregation jedes Mal neu -> Teuer!)

Bonusübungen

1. **WITH CHECK OPTION:** Erstelle eine View für Mitarbeiter der Abteilung 'Sales'. Füge die Option `WITH CHECK OPTION` hinzu. Versuche nun, über diese View einen Mitarbeiter in 'Marketing' einzufügen.
2. **Updatable View:** Versuche, das Gehalt eines Mitarbeiters direkt über die View `v_current_employees` zu ändern. Funktioniert das? Warum (nicht)?
3. **Index auf Virtual Column:** Erstelle einen Index auf die `full_name` Spalte aus Aufgabe 2 (falls sie STORED ist, oder direkt auf den Ausdruck bei MySQL 8+). Prüfe mit EXPLAIN, ob eine Suche nach dem vollen Namen nun den Index nutzt.
4. **Security Definer View:** Erstelle eine View, die auf eine Tabelle zugreift, auf die der aktuelle User keinen Zugriff hat. Definiere die View mit `SQL SECURITY DEFINER`. Kann der User die View lesen?
5. **View auf View:** Erstelle eine View `v_managers`, die auf `dept_manager` basiert. Erstelle dann eine View `v_current_managers`, die auf `v_managers` basiert und nur aktuelle filtert.
6. **Algorithm Merge vs. Temptable:** Zwingen MySQL bei einer View-Erstellung, den Algorithmus `TEMPTABLE` zu nutzen (`CREATE ALGORITHM=TEMPTABLE VIEW ...`). Prüfe mit `EXPLAIN`, wie sich der Zugriff ändert.
7. **Generated Column (Date):** Erstelle eine Generated Column `hire_year` in `employees`, die nur das Jahr des Einstellungsdatums enthält. Indiziere diese Spalte.
8. **Check Option Cascaded:** Erstelle eine View mit `WITH CASCADED CHECK OPTION`, die auf einer anderen View basiert. Versuche, Daten einzufügen, die die Bedingung der Basis-View verletzen.

9. **JSON in Virtual Column:** Extrahiere einen Wert aus einem JSON-Feld in eine Virtual Column und indiziere diese. (Nutze die Tabelle aus der JSON-Übung).
10. **Materialized View Workaround:** Da MySQL keine nativen Materialized Views hat: Erstelle eine Tabelle `mv_dept_stats` und fülle sie mit dem Ergebnis einer komplexen Aggregation. Schreibe ein Event, das diese Tabelle alle 5 Minuten aktualisiert.

Übungen: Transaktionen & Locking

Diese Übungen erfordern zwei parallele Verbindungen (z.B. zwei Browser-Tabs in phpMyAdmin oder zwei Terminal-Fenster).

Aufgabe 1: Der sichere Geldtransfer

Szenario: Du willst das Gehalt von Mitarbeiter 10001 um 5000 reduzieren und Mitarbeiter 10002 um 5000 erhöhen.

1. Starte eine Transaktion (`START TRANSACTION`).
2. Führe das erste Update aus.
3. Prüfe in einer **anderen** Session, ob sich der Wert schon geändert hat. (Sollte nicht!)
4. Führe das zweite Update aus.
5. `COMMIT`.
6. Prüfe erneut in der anderen Session.

Aufgabe 2: Dirty Read Simulation

Ziel: Verstehe `READ UNCOMMITTED`.

1. **Session A:** Setze Isolation Level: `SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED`.
2. **Session B:** Starte Transaktion, ändere einen Namen in `employees` (z.B. "Georgi" -> "Schorsch"). **Kein Commit!**
3. **Session A:** Lese den Datensatz. Siehst du "Schorsch"? (Ja!)
4. **Session B:** `ROLLBACK`.
5. **Session A:** Lese erneut. Was siehst du jetzt?

Aufgabe 3: Deadlock provozieren

Ziel: Bringe die Datenbank dazu, einen Fehler 1213 zu werfen.

1. **Session A:** `START TRANSACTION; UPDATE salaries SET salary = salary + 1 WHERE emp_no = 10001;`
2. **Session B:** `START TRANSACTION; UPDATE salaries SET salary = salary + 1 WHERE emp_no = 10002;`
3. **Session A:** `UPDATE salaries SET salary = salary + 1 WHERE emp_no = 10002;`
(Wartet...)
4. **Session B:** `UPDATE salaries SET salary = salary + 1 WHERE emp_no = 10001;`
(Boom!)

Aufgabe 4: Pessimistic Locking

Ziel: Verhindere, dass jemand anderes Daten ändert, während du sie liest.

1. **Session A:** `START TRANSACTION; SELECT * FROM employees WHERE emp_no = 10001 FOR UPDATE;`
2. **Session B:** Versuche, den Namen von 10001 zu ändern. Was passiert?
3. **Session A:** `COMMIT`. Was passiert jetzt bei B?

Bonusübungen

1. **Savepoints:** Starte eine Transaktion, setze einen `SAVEPOINT my_save`. Ändere Daten. Mache einen `ROLLBACK TO SAVEPOINT my_save`. Prüfe, ob die Änderungen weg sind, die Transaktion aber noch offen ist.
2. **Lock Wait Timeout:** Setze in einer Session `SET SESSION innodb_lock_wait_timeout = 1;`. Provoziere einen Lock Wait (wie in Aufgabe 3 oder 4). Beobachte, wie schnell der Fehler kommt.
3. **Phantom Read:** Versuche, in zwei Sessions ein "Phantom Read" Phänomen nachzustellen (Session A liest alle Mitarbeiter, Session B fügt einen neuen hinzu, Session A liest erneut). Vergleiche das Verhalten von `REPEATABLE READ` (Standard) und `READ COMMITTED`.

4. **Locking Read (Share Mode):** Nutze `SELECT ... LOCK IN SHARE MODE` (oder `FOR SHARE` in neueren Versionen). Versuche in einer anderen Session, den Datensatz zu lesen (sollte gehen) und zu schreiben (sollte blockieren).
5. **Foreign Key Deadlock:** Erstelle zwei Tabellen mit gegenseitigen Foreign Keys. Versuche, in zwei Transaktionen über Kreuz Daten einzufügen, um einen Deadlock zu provozieren.
6. **Serializable:** Setze das Isolation Level auf `SERIALIZABLE`. Prüfe, wie sich das Verhalten bei einfachen SELECTs ändert (werden sie zu Locking Reads?).
7. **Autocommit:** Deaktiviere `autocommit` (`SET autocommit=0`). Führe Updates durch. Beende die Session ohne Commit. Sind die Daten weg?
8. **Information Schema Locks:** Während eine Transaktion blockiert (z.B. durch `FOR UPDATE`), prüfe in einer dritten Session die Tabelle `information_schema.INNODB_TRX` oder `performance_schema.data_locks`, um den Lock zu sehen.
9. **Skipped Locked:** Nutze `SELECT ... FOR UPDATE SKIP LOCKED`. Starte zwei Sessions, die beide versuchen, "die nächsten 5 offenen Aufgaben" zu holen. Sie sollten unterschiedliche Zeilen bekommen.
10. **Consistent Snapshot:** Starte eine Transaktion mit `START TRANSACTION WITH CONSISTENT SNAPSHOT`. Ändere Daten in einer anderen Session. Prüfe, ob deine Transaktion den alten Stand sieht ("Time Travel").

Übungen: Funktionen, Window Functions & JSON

Aufgabe 1: Window Functions (Ranking)

Ziel: Erstelle eine Rangliste der bestbezahlten Mitarbeiter **pro Abteilung**.

- **Tabelle:** salaries, dept_emp
- **Funktion:** RANK() oder DENSE_RANK()
- **Partitionierung:** Nach dept_no
- **Sortierung:** Nach salary DESC
- **Ausgabe:** dept_no, emp_no, salary, rank

Aufgabe 2: Window Functions (Lag)

Ziel: Analysiere die Gehaltsentwicklung von Mitarbeiter 10001.

- Zeige für jeden Gehaltseintrag an, wie hoch das **vorherige** Gehalt war und wie groß die Differenz ist.
- **Funktion:** LAG(salary) OVER (ORDER BY from_date)

Aufgabe 3: JSON Basics

Wir haben keine JSON-Tabelle in employees, also erstellen wir eine temporäre zum Üben.

```
CREATE TEMPORARY TABLE user_configs (  
    id INT PRIMARY KEY,  
    config JSON  
);  
  
INSERT INTO user_configs VALUES
```

```
(1, '{"theme": "dark", "notifications": {"email": true, "sms": false}}'),  
(2, '{"theme": "light", "notifications": {"email": false, "sms": true}}');
```

1. Finde alle User, die das "dark" Theme nutzen.
2. Finde alle User, die SMS-Benachrichtigungen aktiviert haben.
3. Ändere bei User 1 das Theme auf "light" (nutze `JSON_SET` oder `JSON_REPLACE`).

Aufgabe 4: Eigene Funktion (Stored Function)

Ziel: Schreibe eine Funktion `get_years_employed(p_emp_no INT)`.

- Sie soll zurückgeben, wie viele Jahre ein Mitarbeiter schon in der Firma ist.
- **Logik:** Differenz zwischen `hire_date` und `NOW()` (oder `to_date` bei Ehemaligen).
- **Tipp:** `TIMESTAMPDIFF(YEAR, start, end)` ist hilfreich.

Bonusübungen

1. **JSON_TABLE:** Nutze `JSON_TABLE`, um das JSON aus Aufgabe 3 temporär in eine relationale Struktur (Spalten: `theme`, `email_notif`) zu verwandeln und abzufragen.
2. **NTILE:** Teile alle Mitarbeiter basierend auf ihrem aktuellen Gehalt in 4 Gruppen (Quartile) ein. Wer gehört zu den Top 25%?
3. **Datums-Rechnung:** Berechne das genaue Alter aller Mitarbeiter in Tagen (`DATEDIFF`).
4. **JSON_MERGE:** Füge zwei JSON-Objekte zusammen (z.B. Default-Config und User-Config). Nutze `JSON_MERGE_PATCH` (oder `PRESERVE`).
5. **JSON Validation:** Nutze `JSON_VALID`, um zu prüfen, ob ein String valides JSON enthält.
6. **JSON Search:** Finde den Pfad zu einem bestimmten Wert in einem JSON-Dokument mit `JSON_SEARCH`.

7. **Regular Expressions:** Finde alle Mitarbeiter, deren Nachname mit einem Vokal beginnt und mit einem Vokal endet (`REGEXP`).
8. **Coalesce:** Nutze `COALESCE`, um NULL-Werte in einer Abfrage durch einen Standardwert (z.B. "N/A") zu ersetzen.
9. **Cast & Convert:** Konvertiere einen String "123.45" explizit in eine DECIMAL-Zahl und rechne damit.
10. **Random Samples:** Wähle zufällig 5 Mitarbeiter aus (`ORDER BY RAND() LIMIT 5`). Diskutiere die Performance bei großen Tabellen.

Übungen: Stored Procedures, Triggers & Events

Aufgabe 1: Die "Hire" Prozedur

Ziel: Vereinfache das Einstellen neuer Mitarbeiter.

- Erstelle eine Procedure `hire_employee`.
- **Parameter:** `first_name`, `last_name`, `gender`, `birth_date`, `dept_name`, `title`, `salary`.
- **Logik:**
 - i. Neue `emp_no` generieren (`MAX + 1`).
 - ii. Insert in `employees`.
 - iii. Insert in `dept_emp` (mit `dept_no` lookup).
 - iv. Insert in `titles`.
 - v. Insert in `salaries`.
 - vi. Alles in einer Transaktion!

Aufgabe 2: Salary Protection Trigger

Ziel: Verhindere, dass das Gehalt gesenkt wird.

- Erstelle einen `BEFORE UPDATE` Trigger auf `salaries`.
- Wenn `NEW.salary < OLD.salary`, wirf einen Fehler (`SIGNAL SQLSTATE '45000'`).

Aufgabe 3: Audit Event

Ziel: Lösche alte Audit-Logs automatisch.

- Nimm an, wir haben eine Tabelle `audit_log` mit `created_at`.
- Erstelle ein Event, das jede Nacht läuft und Einträge löscht, die älter als 1 Jahr sind.

Aufgabe 4: Cursor Loop

Ziel: Berechne einen Bonus für alle Mitarbeiter.

- Erstelle eine Prozedur `calculate_bonuses`.
- Nutze einen Cursor, um über alle Mitarbeiter zu iterieren.
- Wenn der Mitarbeiter länger als 10 Jahre dabei ist -> Bonus in eine separate Tabelle `bonuses` schreiben.

Bonusübungen

1. **OUT Parameter:** Schreibe eine Prozedur `get_dept_stats`, die für eine gegebene `dept_no` drei Werte zurückgibt (OUT Parameter): Min-Gehalt, Max-Gehalt und Durchschnittsgehalt.
2. **Security Context:** Erstelle eine Prozedur, die auf eine Tabelle zugreift, auf die der normale User keinen Zugriff hat. Nutze `SQL SECURITY DEFINER`, damit der User die Prozedur trotzdem erfolgreich ausführen kann.
3. **Show Create:** Nutze `SHOW CREATE PROCEDURE hire_employee` (oder den Namen deiner Prozedur), um dir den Quellcode und die Einstellungen der Prozedur anzeigen zu lassen.
4. **Error Handling (Resignal):** Fange eine Exception in einem Handler, logge sie in eine Tabelle und wirf sie dann erneut (`RESIGNAL`), damit der Aufrufer auch merkt, dass etwas schiefging.
5. **Dynamic SQL (Prepared):** Schreibe eine Prozedur, die einen Tabellennamen als Parameter nimmt und die Anzahl der Zeilen in dieser Tabelle zurückgibt (Dynamisches SQL nötig!).
6. **Deterministic:** Markiere eine Funktion als `DETERMINISTIC` (wenn sie es ist). Prüfe, ob das Auswirkungen auf die Performance hat (Query Cache, Optimizer).

7. **Trigger Cascade:** Erstelle einen Trigger A, der eine Tabelle ändert, auf der Trigger B liegt. Prüfe, ob Trigger B feuert.
8. **Event Scheduling:** Erstelle ein Event, das *einmalig* in 5 Minuten läuft und dann automatisch gelöscht wird (`ON COMPLETION NOT PRESERVE`).
9. **Stored Procedure calling Stored Procedure:** Lagere Teil-Logik in eine Sub-Prozedur aus und rufe diese aus der Haupt-Prozedur auf.
10. **User Variables:** Nutze User-Variablen (`@my_var`), um Werte zwischen zwei Prozedur-Aufrufen innerhalb derselben Session zu teilen.

Übungen: Tuning, Indexing & Profiling

Aufgabe 1: The Slow Query

Führe folgende Query aus (ohne Index):

```
SELECT * FROM salaries WHERE salary = 60117;
```

1. Miss die Zeit.
2. Nutze `EXPLAIN`. Welchen `type` siehst du? (`ALL?`)
3. Erstelle einen Index auf `salary`.
4. Miss die Zeit erneut.
5. Nutze `EXPLAIN` erneut. Welchen `type` siehst du jetzt? (`ref?`)

Aufgabe 2: Composite Index

Wir suchen oft nach Mitarbeitern anhand von Vor- und Nachname.

```
SELECT * FROM employees WHERE last_name = 'Facello' AND first_name = 'Georgi';
```

1. Erstelle einen Index `idx_name` auf `(last_name, first_name)`.
2. Prüfe mit `EXPLAIN`, ob er genutzt wird.
3. Prüfe folgende Query: `SELECT * FROM employees WHERE first_name = 'Georgi';`.
Wird der Index genutzt? Warum (nicht)?

Aufgabe 3: Sorting Optimization

```
SELECT * FROM employees ORDER BY hire_date LIMIT 10;
```

1. Prüfe `EXPLAIN`. Siehst du `Using filesort`?
2. Wie bekommst du das `filesort` weg? (Tipp: Index auf `hire_date`).

Aufgabe 4: App-Side Profiling (Bun)

Schreibe ein kleines Bun-Skript.

1. Führe eine Query aus, die 100.000 Zeilen lädt (z.B. `SELECT * FROM salaries LIMIT 100000`).
2. Miss die Zeit für die Query (`await db...`).
3. Miss die Zeit für das Verarbeiten (z.B. durchloopen und summieren).
4. Was dauert länger? Transfer oder Verarbeitung?

Bonusübungen

1. **Invisible Index:** Mache einen deiner Indizes "unsichtbar" (`ALTER TABLE ... ALTER INDEX ... INVISIBLE`). Prüfe mit `EXPLAIN`, ob er nun ignoriert wird.
2. **Prefix Index:** Die Spalte `last_name` ist VARCHAR. Erstelle einen Index, der nur die ersten 3 Zeichen indiziert. Prüfe die Größe des Index im Vergleich zum vollen Index (Theorie).
3. **Force Index:** Zwinge MySQL bei einer Query mittels `FORCE INDEX (index_name)`, einen bestimmten Index zu nutzen, auch wenn der Optimizer einen anderen (oder Full Table Scan) bevorzugen würde. Vergleiche die Performance.
4. **Covering Index:** Erstelle eine Query, die *nur* Spalten abfragt, die im Index enthalten sind. Prüfe im EXPLAIN, ob `Using index` (nicht `Using index condition`!) steht. Das ist extrem schnell.
5. **Index Merge:** Schreibe eine Query mit `WHERE a=1 OR b=2`. Wenn beide Spalten indiziert sind, kann MySQL einen "Index Merge" machen. Prüfe das EXPLAIN.
6. **Analyze Table:** Führe `ANALYZE TABLE employees` aus. Das aktualisiert die Statistiken für den Optimizer. Wann sollte man das tun?
7. **Optimizer Hints:** Nutze einen Optimizer Hint (z.B. `/*+ MAX_EXECUTION_TIME(1000) */`), um eine Query abubrechen, wenn sie zu lange dauert.

8. **Duplicate Index:** Suche (manuell oder per Tool) nach redundanten Indizes in der Datenbank (z.B. Index A auf (col1) und Index B auf (col1, col2)). Index A ist redundant.
9. **Cardinality:** Prüfe die Kardinalität (Anzahl eindeutiger Werte) deiner Indizes mit `SHOW INDEX FROM table`. Warum ist eine hohe Kardinalität gut für Indizes?
10. **Performance Schema:** (Fortgeschritten) Aktiviere einen Consumer im Performance Schema und versuche, Details zu deiner letzten Query dort zu finden.

Übung 07: GIS (Geodaten)

Szenario

Wir bauen eine einfache "Store Locator" Datenbank für eine Kette von E-Scooter Ladestationen.

Aufgaben

1. Tabelle erstellen

Erstelle eine Tabelle `charging_stations` mit:

- `id` (INT, PK, AI)
- `name` (VARCHAR)
- `location` (POINT, SRID 4326, NOT NULL)
- Einem `SPATIAL INDEX` auf `location`.

2. Daten einfügen

Füge folgende Stationen ein (Achtung: Reihenfolge Longitude/Latitude beachten!):

- "Alexanderplatz": 13.413215, 52.521918
- "Brandenburger Tor": 13.377704, 52.516275
- "Hauptbahnhof": 13.369402, 52.525084
- "Potsdamer Platz": 13.376198, 52.509669

3. Distanz berechnen

Berechne die Distanz (in Metern) vom "Alexanderplatz" zum "Brandenburger Tor".
Nutze `ST_Distance_Sphere`.

4. Umkreissuche

Finde alle Stationen im Umkreis von 2000 Metern um den "Checkpoint Charlie" (13.390391, 52.507443). Gib Name und Distanz aus, sortiert nach Distanz.

5. Bonus: Polygon

Definiere ein Polygon für den "Tiergarten" (grob) und finde alle Stationen, die *innerhalb* dieses Polygons liegen.

- Tipp: `ST_Within`
- Polygon Punkte (grob):
 - 13.33, 52.51
 - 13.38, 52.51
 - 13.38, 52.52
 - 13.33, 52.52
 - 13.33, 52.51 (Polygon muss geschlossen sein!)

6. Bonus: GeoJSON

Gib die `location` aller Stationen als GeoJSON zurück (`ST_AsGeoJSON`).

Der Umzug: Migration zu MariaDB

In dieser Übung simulieren wir den Umzug einer produktiven Datenbank von MySQL zu MariaDB. Da beide Systeme das gleiche Protokoll sprechen, ist der logische Export/Import (Dump) oft der sicherste Weg für kleinere bis mittlere Datenbanken.

Szenario

Wir haben eine laufende MySQL-Instanz (mit der `employees` Datenbank) und wollen diese in eine frische MariaDB-Instanz umziehen. Zusätzlich existiert eine `app_db`, die wir ebenfalls betrachten könnten, aber wir fokussieren uns erst auf `employees`.

Aufgabe 1: Der "perfekte" Export

Nutze `mysqldump` (oder `mariadb-dump`), um die Datenbank `employees` zu exportieren. Ein einfacher Dump (`mysqldump employees > dump.sql`) reicht in Produktion **nicht** aus! Warum?

1. Er blockiert Tabellen (bei MyISAM).
2. Er garantiert keinen consistent snapshot (bei InnoDB, wenn sich Daten während des Dumps ändern).
3. Er vergisst vielleicht Stored Procedures oder Events.
4. Er hat Probleme mit Binary Blobs, wenn das Encoding nicht stimmt.

ToDo

Erstelle einen Befehl, der all diese Probleme löst.

▼ Lösung anzeigen

```
mysqldump -u root -p \  
--single-transaction \  
--routines --triggers --events \  
--hex-blob \  
--default-character-set=utf8mb4 \  
--quick \  
employees > employees_migration.sql
```

Erklärung der Flags:

- `--single-transaction`: **Der wichtigste Flag für InnoDB!** Startet eine Transaktion, liest den Snapshot konsistent, ohne Tabellen zu sperren. (Ohne das werden bei MyISAM Tabellen gelockt!).
- `--routines --triggers --events`: Standardmäßig werden Stored Procs & Events oft nicht exportiert. Wir brauchen sie aber!
- `--hex-blob`: Exportiert Binärdaten (`BINARY`, `BLOB`) als Hex-Strings (z.B. `0xA1B2...`). Verhindert Encoding-Salat.
- `--quick`: Liest Row-by-Row statt die ganze Tabelle in den RAM zu laden. Wichtig bei großen Tabellen.
- `--default-character-set=utf8mb4`: Erzwingt, dass der Output UTF8mb4 ist (kein Latin1-Fallback).

Aufgabe 2: Der Import

Nun spielen wir die Daten in die neue Instanz ein.

ToDo

Importiere `employees_migration.sql` in die neue Datenbank. Was muss vorher existieren?

▼ Lösung anzeigen

Die Datenbank muss leer existieren (falls sie nicht im Dump mit `CREATE DATABASE` steht):

```
# 1. DB anlegen
mysql -u root -p -e "CREATE DATABASE IF NOT EXISTS employees CHARACTER SET
utf8mb4 COLLATE utf8mb4_unicode_ci;"

# 2. Importieren
mysql -u root -p employees < employees_migration.sql
```

Best Practices beim Import:

- **Netzwerk:** Pipe nutzen (`mysqldump ... | mysql ...`), um Disk-IO zu sparen, wenn beide Server erreichbar sind.
- **Fehler:** Bei riesigen Dumps `mysql --force` nutzen? Nein! Besser: Skript stoppt bei Fehler sofort.
- **Performance:** Für riesige Imports: `ALTER TABLE ... DISABLE KEYS` (nur MyISAM) oder `autocommit=0` und `unique_checks=0` im Dump-Header (macht `mysqldump` meist automatisch).

Aufgabe 3: Best Practices & Diskussion

Diskutiert folgende Fragen:

1. Warum kopieren wir nicht einfach den Daten-Ordner (`/var/lib/mysql`)?

- *Antwort:* Funktioniert nur, wenn Versionen identisch sind. MySQL 8.0 File-Format ist **nicht** kompatibel mit MariaDB (andere System-Tabellen, Log-Formate).

2. Was, wenn die DB 500 GB groß ist?

- *Antwort:* `mysqldump` ist Single-Threaded -> zu langsam.
- Alternative: **MySQL Shell Dump** (Parallel!) oder **MyDumper** (Parallel).
- Oder: Replikation aufsetzen und dann umschwenken (Minimal Downtime).

3. MariaDB vs MySQL Kompatibilität:

- Achte auf JSON! MySQL speichert JSON binär, MariaDB als Text (Check Constraint). Ein Dump von MySQL 8 mit JSON könnte in MariaDB Warnungen werfen oder Syntax-Fehler bei generierten Columns haben.

Replikation in MySQL und MariaDB

Warum eigentlich Replikation?

WAS WÄRE, WENN...

Was passiert, wenn die zentrale Datenbank im Unternehmen ausfällt?

Wenn eine Datenbank der einzige zentrale Speicherpunkt ist, führt ein Ausfall zu:

- Produktionsstillstand
- Datenverlust
- Ausfallkosten
- Imageschäden bei Kunden

Replikation ist eine grundlegende Technik, um Systeme **robuster**, **skalierbarer** und **ausfallsicherer** zu gestalten.

Ziele der Replikation

Replikation verfolgt mehrere betriebliche und technische Ziele:

- **Hochverfügbarkeit (High Availability)**
Daten bleiben auch bei Serverausfall erreichbar.
- **Lastverteilung (Load Balancing)**
Leseanfragen können auf mehrere Server verteilt werden.
- **Datensicherheit & Redundanz**
Kopien der Daten schützen vor Hardware- oder Softwarefehlern.
- **Entlastung des Primärsystems**
Backups, Reports und Analysen laufen auf Replikas, nicht auf dem Master.

- **Geografische Verteilung**

Systeme an unterschiedlichen Standorten erhalten synchronisierte Datenbestände.

 MERKSATZ

Replikation bedeutet nicht Konsistenz, sondern Redundanz.

Wie funktioniert Replikation in MySQL und MariaDB?

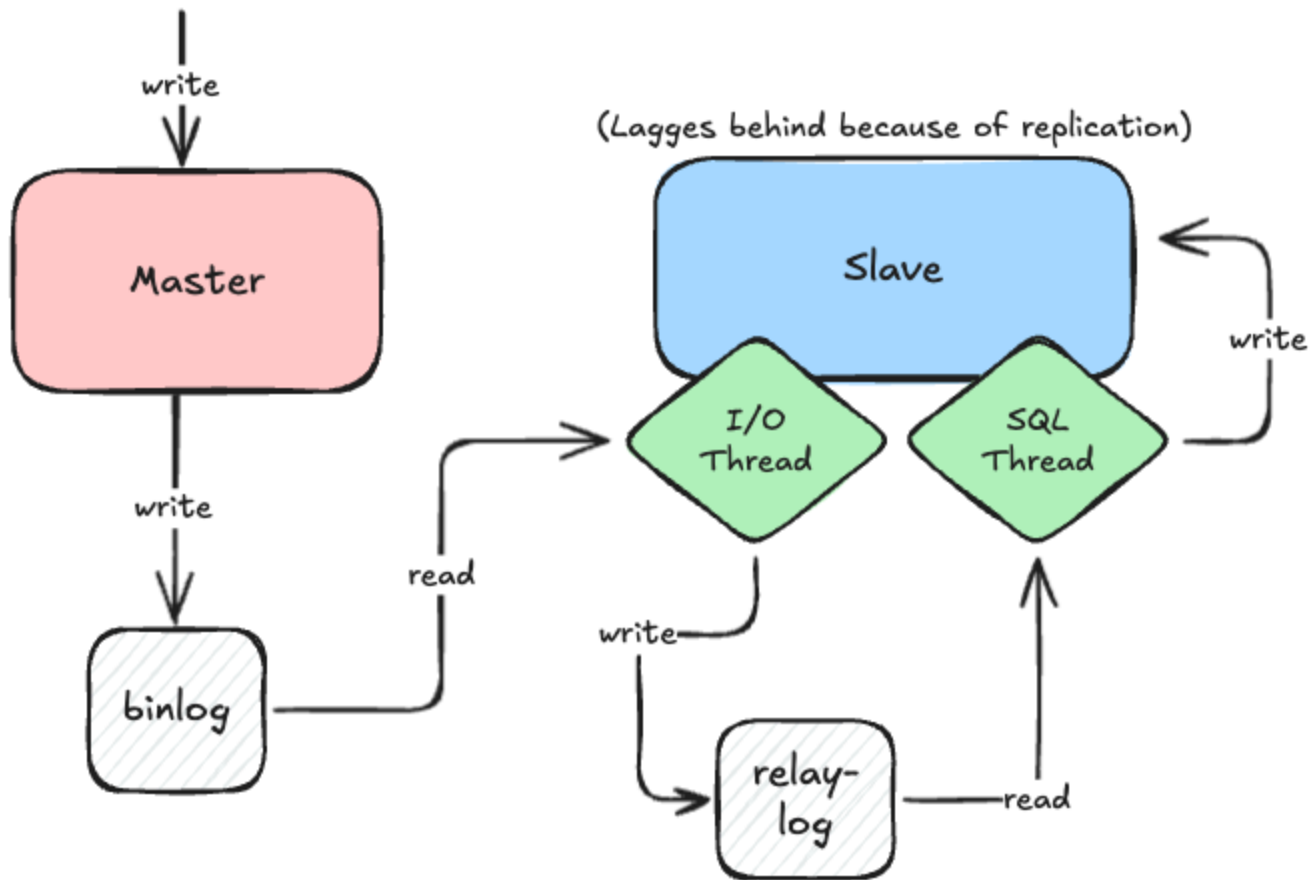
Replikation ist das kontinuierliche Kopieren von Änderungen vom **Master** auf einen oder mehrere **Slaves**.

Dabei werden nicht komplette Datenbanken übertragen, sondern nur die **Transaktionen**, die am Master ausgeführt wurden.

Ablauf im Detail

1. Der **Master** schreibt jede Änderung in das **Binary Log** (**binlog**).
2. Der **Slave** liest die Binlogs über seinen **I/O-Thread** und speichert sie im **Relay Log**.
3. Der **SQL-Thread** auf dem Slave führt die Anweisungen in derselben Reihenfolge aus.
4. Das Ergebnis ist eine zeitlich verzögerte, aber inhaltlich gleiche Kopie der Datenbank.

INSERT INTO users VALUES....



Der Master akzeptiert Schreiboperationen, die Slaves folgen passiv nach. Das bedeutet, die Daten auf den Slaves sind **repliziert**, aber nicht zwangsläufig **zeitgleich konsistent**.

Eigenschaften

- **Asynchrone Übertragung:** Der Master wartet nicht auf Bestätigung der Slaves.
- **Zeitverzögerte Ausführung:** Slaves hängen je nach Systemlast hinterher.
- **Lesbare Replikas:** Slaves können Leseanfragen verarbeiten, um den Master zu entlasten.
- **Ein Master, mehrere Slaves:** Typisches Muster für Web- oder Reporting-Umgebungen.

💡 PRAXIS

Ein Slave kann hinterherhinken.

Prüfe mit:

```
SHOW SLAVE STATUS\G
```

Der Wert Seconds_Behind_Master zeigt die Verzögerung in Sekunden.

Warum Galera Cluster?

Grenzen der Standard-Replikation

Wir haben gesehen, wie einfach sich eine Master-Slave-Replikation aufsetzen lässt. Sie ist ein bewährtes Mittel für:

- **Backups** (auf dem Slave)
- **Analysen/Reporting** (ohne den Master zu belasten)
- **Einfache Skalierung von Lesezugriffen**

Doch in modernen, hochverfügbaren Umgebungen stoßen wir schnell auf harte Grenzen:

1. Datenverlust bei Ausfall (Asynchronität)

Da die Replikation standardmäßig **asynchron** läuft, bestätigt der Master eine Transaktion (`COMMIT`), *bevor* der Slave sie empfangen hat. Fällt der Master in genau diesem Moment aus (z.B. Stromausfall, Crash), sind die letzten Transaktionen **verloren**.



In der Standard-Replikation ist der **Recovery Point Objective (RPO)** immer größer als Null. Datenverlust ist im "Worst Case" nicht ausgeschlossen.

2. Komplexer Failover

Wenn der Master ausfällt, müssen wir:

1. Erkennen, dass er wirklich endgültig ausgefallen ist.
2. Den aktuellsten Slave finden (wer hat die meisten Binlogs?).
3. Diesen Slave zum neuen Master befördern.
4. Alle anderen Slaves auf den neuen Master umkonfigurieren (`CHANGE MASTER TO...`).
5. Die Applikation auf die IP des neuen Masters schwenken.

Das ist manuell fehleranfällig und langsam. Tools wie *Orchestrator* oder *MHA* können das automatisieren, erhöhen aber die Komplexität der Infrastruktur enorm.

3. Keine Schreib-Skalierung

Egal wie viele Slaves wir hinzufügen: **Geschrieben wird nur auf dem Master.** Wenn der Master bei 100% CPU/IO-Last ist, hilft kein weiterer Slave. Wir sind vertikal limitiert ("Scale Up").

Die Lösung: Galera Cluster

Galera Cluster ist eine **synchrone Multi-Master** Replikationstechnologie. Sie löst genau diese Probleme.

Synchronität (Virtuell)

Galera garantiert, dass eine Transaktion entweder auf **allen** Knoten committed wird oder auf **keinem**. Wenn ein Knoten ausfällt, haben die anderen garantiert den gleichen Datenstand.

KEIN DATENVERLUST

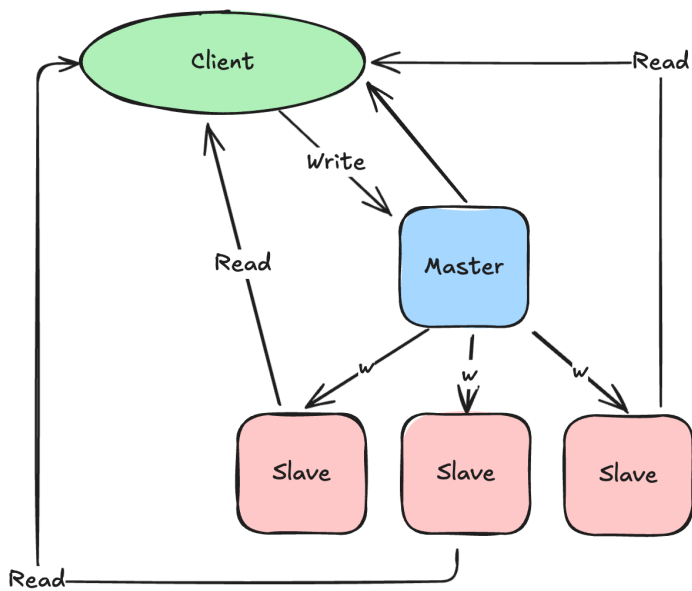
Mit Galera ist ein **RPO von 0** möglich. Keine Transaktion geht bei einem Node-Crash verloren.

Multi-Master (Active-Active)

Jeder Knoten kann Lese- **und** Schreibzugriffe verarbeiten.

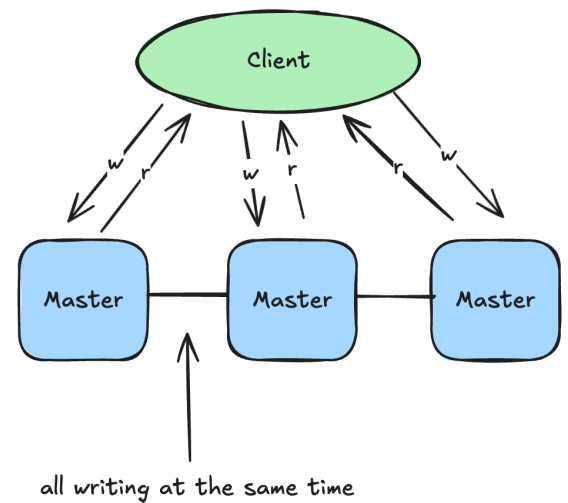
- Keine Unterscheidung mehr zwischen "Master" und "Slave".
- Clients können sich mit jedem Knoten verbinden.
- Fällt ein Knoten aus, nutzen die Clients einfach einen anderen.

Classic Replication



Eventual Consistency!

Galera Cluster



Strong Consistency!

Automatische Node-Provisionierung

Ein neuer (oder reparierter) Knoten tritt dem Cluster bei und holt sich die Daten **automatisch** von einem Spender (Donor). Kein manuelles Dump-Erstellen, kein Hantieren mit Binlog-Positionen.

Vergleich: Standard vs. Galera

Feature	Standard Replikation	Galera Cluster
Replikationsart	Asynchron	Synchron (Zertifizierungsbasiert)
Datenverlust	Möglich (bei Master-Crash)	Ausgeschlossen (bei Node-Crash)
Failover	Komplex / Manuell / Externe Tools	Automatisch / Transparent
Schreiben	Nur auf Master (1 Node)	Auf allen Nodes möglich

Feature	Standard Replikation	Galera Cluster
Latenz	Gering (Schreiben ist lokal schnell)	Etwas höher (Netzwerk-Roundtrip nötig)
Konflikte	Nicht möglich (nur einer schreibt)	Möglich (Zertifizierungsfehler bei gleichen Rows)

Wann Galera?

Galera ist die richtige Wahl, wenn:

- **Datenverlust inakzeptabel** ist (Banking, E-Commerce).
- **Hochverfügbarkeit** (24/7) gefordert ist.
- Manuelle Failover-Prozesse eliminiert werden sollen.

Installation PMM

1. user zu docker group: `sudo usermod -aG docker $USER`
2. Maria DBBare Metal install: <https://mariadb.com/docs/server/server-management/install-and-upgrade-mariadb/installing-mariadb/binary-packages/installing-mariadb-deb-files>

```
curl -sS https://downloads.mariadb.com/MariaDB/mariadb_repo_setup | sudo bash
sudo apt update
sudo apt install mariadb-server mariadb-client
```

```
curl -fsSL https://www.percona.com/get/pmm | /bin/bash
```

3. PMM Client Installation: https://docs.percona.com/percona-monitoring-and-management/3/install-pmm/install-pmm-client/package_manager.html

```
wget https://repo.percona.com/apt/percona-release_latest.generic_all.deb
sudo dpkg -i percona-release_latest.generic_all.deb
sudo percona-release enable pmm3-client
```

```
sudo apt update
sudo apt install -y pmm-client
```

```
pmm-admin config --server-insecure-tls \
--server-url=https://admin:okapi23@127.0.0.1:443
```

<https://docs.percona.com/percona-monitoring-and-management/3/install-pmm/install-pmm-client/connect-database/mysql/mysql.html>

peroformanceschema config: https://docs.percona.com/percona-monitoring-and-management/3/install-pmm/install-pmm-client/connect-database/mysql/mysql.html#__tabbed_3_2

```
CREATE USER 'pmm'@'localhost' IDENTIFIED BY 'okapi23' WITH MAX_USER_CONNECTIONS
10;
```

```
GRANT SELECT, PROCESS, REPLICATION CLIENT, RELOAD ON *.* TO 'pmm'@'localhost';
```

MySQL Cheatsheet

Eine schnelle Referenz für die wichtigsten SQL-Befehle und Funktionen.

1. Grundlegende Abfragen (SELECT)

Daten abrufen ist die häufigste Operation.

```
-- Alle Spalten
SELECT * FROM employees;

-- Spezifische Spalten
SELECT first_name, last_name FROM employees;

-- Duplikate entfernen
SELECT DISTINCT title FROM titles;

-- Limitieren (Pagination)
SELECT * FROM employees LIMIT 10 OFFSET 20;
```

2. Filtern (WHERE)

Daten einschränken.

```
-- Exakter Match
SELECT * FROM employees WHERE gender = 'M';

-- Vergleichsoperatoren
SELECT * FROM salaries WHERE salary > 50000;
SELECT * FROM salaries WHERE salary BETWEEN 40000 AND 60000;

-- Liste von Werten
SELECT * FROM employees WHERE emp_no IN (10001, 10002, 10003);

-- Muster (LIKE)
-- % = beliebig viele Zeichen, _ = genau ein Zeichen
```

```

SELECT * FROM employees WHERE first_name LIKE 'Mar%'; -- Startet mit Mar
SELECT * FROM employees WHERE last_name LIKE '%son'; -- Endet mit son

-- NULL Prüfung
SELECT * FROM employees WHERE birth_date IS NOT NULL;

```

3. Sortieren (ORDER BY)

```

-- Aufsteigend (Standard)
SELECT * FROM employees ORDER BY hire_date ASC;

-- Absteigend
SELECT * FROM employees ORDER BY hire_date DESC;

-- Mehrere Spalten
SELECT * FROM employees ORDER BY last_name ASC, first_name ASC;

```

4. Aggregation & Gruppierung

Daten zusammenfassen und analysieren.

Aggregat-Funktionen

Funktion	Beschreibung
COUNT(*)	Zählt alle Zeilen (inkl. NULL).
COUNT(col)	Zählt Zeilen, wo col nicht NULL ist.
SUM(col)	Summiert Werte.
AVG(col)	Berechnet den Durchschnitt.
MAX(col)	Findet den größten Wert.

Funktion	Beschreibung
<code>MIN(col)</code>	Findet den kleinsten Wert.

```
SELECT COUNT(*) FROM employees;
SELECT AVG(salary) FROM salaries WHERE to_date > NOW();
```

GROUP BY

Gruppieren Zeilen mit gleichen Werten, um Aggregationen pro Gruppe durchzuführen.

```
-- Anzahl Mitarbeiter pro Titel
SELECT title, COUNT(*)
FROM titles
WHERE to_date > NOW()
GROUP BY title;

-- Durchschnittsgehalt pro Abteilung
SELECT dept_no, AVG(salary)
FROM dept_emp
JOIN salaries USING (emp_no)
GROUP BY dept_no;
```

HAVING

Filtert **nach** der Aggregation (wo `WHERE` nicht funktioniert).

```
-- Nur Abteilungen mit mehr als 10.000 Mitarbeitern
SELECT dept_no, COUNT(*) as emp_count
FROM dept_emp
GROUP BY dept_no
HAVING emp_count > 10000;
```

5. Daten Manipulation (DML)

```
-- Einfügen
INSERT INTO employees (emp_no, birth_date, first_name, last_name, gender,
hire_date)
VALUES (999999, '1990-01-01', 'Max', 'Mustermann', 'M', NOW());

-- Aktualisieren
UPDATE salaries
SET salary = salary * 1.05
WHERE emp_no = 10001 AND to_date > NOW();

-- Löschen
DELETE FROM employees WHERE emp_no = 999999;
```

6. Nützliche Funktionen

String

- `CONCAT(a, ' ', b)`: Verbindet Strings.
- `UPPER(s)` / `LOWER(s)`: Groß-/Kleinschreibung.
- `LENGTH(s)`: Länge in Bytes (`CHAR_LENGTH` für Zeichen).
- `SUBSTRING(s, start, len)`: Teilstring extrahieren.
- `TRIM(s)`: Leerzeichen entfernen.

Datum & Zeit

- `NOW()`: Aktuelles Datum + Zeit.
- `CURDATE()`: Nur aktuelles Datum.
- `DATEDIFF(end, start)`: Differenz in Tagen.
- `DATE_ADD(date, INTERVAL 1 DAY)`: Datum rechnen.
- `YEAR(date)`, `MONTH(date)`, `DAY(date)`: Teile extrahieren.

Logik

- `COALESCE(val1, val2, ...)`: Gibt den ersten Nicht-NULL Wert zurück.
- `IFNULL(val, default)`: Wenn val NULL ist, nimm default.

7. Common Table Expressions (CTE)

Temporäre "Tabellen" für eine Query (ab MySQL 8.0). Macht komplexe Queries lesbarer.

```
WITH high_earners AS (  
    SELECT emp_no, salary  
    FROM salaries  
    WHERE salary > 100000 AND to_date > NOW()  
)  
SELECT d.dept_name, COUNT(*) as count  
FROM high_earners he  
JOIN dept_emp de ON he.emp_no = de.emp_no  
JOIN departments d ON de.dept_no = d.dept_no  
GROUP BY d.dept_name;
```

8. JSON Support

MySQL unterstützt JSON nativ. Das ist extrem mächtig für flexible Datenstrukturen.

Tabelle erstellen

```
CREATE TABLE user_settings (  
    user_id INT PRIMARY KEY,  
    settings JSON  
);
```

Daten einfügen

JSON muss als valider String übergeben werden.

```
INSERT INTO user_settings (user_id, settings)  
VALUES (1, '{"theme": "dark", "notifications": {"email": true, "sms":  
false}}');
```


Daten abfragen (Extract)

Es gibt zwei Wege: `JSON_EXTRACT` oder den Pfeil-Operator `->` (mit Anführungszeichen) bzw. `->>` (ohne Anführungszeichen).

```
-- Klassisch
SELECT JSON_EXTRACT(settings, '$.theme') FROM user_settings;

-- Kurzform (gibt "dark" mit Quotes zurück)
SELECT settings->'$.theme' FROM user_settings;

-- Unquoted (gibt dark ohne Quotes zurück - meistens das, was man will)
SELECT settings->>'$.theme' FROM user_settings;

-- Filtern nach JSON Werten
SELECT * FROM salaries;
```

Daten ändern (Update)

Niemals den String manuell manipulieren! Nutze die JSON-Funktionen.

- `JSON_SET`: Fügt hinzu oder überschreibt.
- `JSON_REPLACE`: Überschreibt nur existierende Keys.
- `JSON_INSERT`: Fügt nur neue Keys hinzu.

```
-- JSON_SET: Ändert 'theme' (existiert) UND fügt 'lang' hinzu (neu)
UPDATE user_settings
SET settings = JSON_SET(settings, '$.theme', 'light', '$.lang', 'de')
WHERE user_id = 1;

-- JSON_REPLACE: Ändert NUR 'theme', ignoriert 'new_key' (weil es nicht existiert)
UPDATE user_settings
SET settings = JSON_REPLACE(settings, '$.theme', 'blue', '$.new_key', 'ignored')
WHERE user_id = 1;

-- JSON_INSERT: Fügt NUR 'hero_image' hinzu, ignoriert 'theme' (weil es schon existiert)
UPDATE user_settings
```

```
SET settings = JSON_INSERT(settings, '$.hero_image', 'hero.jpg', '$.theme',  
'red')  
WHERE user_id = 1;
```

Nützliche JSON Funktionen

Funktion	Beschreibung
<code>JSON_KEYS(json)</code>	Gibt alle Keys als Array zurück.
<code>JSON_VALID(str)</code>	Prüft, ob der String valides JSON ist (0 oder 1).
<code>JSON_ARRAY(...)</code>	Erstellt ein JSON Array.
<code>JSON_OBJECT(...)</code>	Erstellt ein JSON Objekt.
<code>JSON_LENGTH(json)</code>	Gibt die Anzahl der Elemente zurück.
<code>JSON_REMOVE(json, path)</code>	Entfernt einen Key.

```
-- Array erstellen: ["a", 1]  
SELECT JSON_ARRAY('a', 1);  
  
-- Objekt erstellen: {"key": "value"}  
SELECT JSON_OBJECT('key', 'value');
```

MariaDB Besonderheiten (JSON)

MariaDB hat keinen nativen JSON-Typ.

```
-- Tabelle erstellen (Constraint ist wichtig!)  
CREATE TABLE user_settings (  
    id INT PRIMARY KEY,  
    settings LONGTEXT CHECK (JSON_VALID(settings))  
);  
  
-- Skalare Werte holen (Typ-sicher)
```

```
SELECT JSON_VALUE(settings, '$.theme') FROM user_settings;
```

```
-- Objekt/Array holen
```

```
SELECT JSON_QUERY(settings, '$.notifications') FROM user_settings;
```

9. Window Functions

Erlauben Berechnungen über Zeilen hinweg, ohne sie zu gruppieren (anders als `GROUP BY`).

Syntax

```
Funktion() OVER (  
    [PARTITION BY spalte]  
    [ORDER BY spalte]  
)
```

Ranking

- `ROW_NUMBER()`: Fortlaufende Nummer (1, 2, 3, 4).
- `RANK()`: Rang mit Lücken bei Gleichstand (1, 2, 2, 4).
- `DENSE_RANK()`: Rang ohne Lücken bei Gleichstand (1, 2, 2, 3).

```
-- Rangliste der Gehälter pro Abteilung
```

```
SELECT  
    dept_no,  
    emp_no,  
    salary,  
    RANK() OVER (PARTITION BY dept_no ORDER BY salary DESC) as rang  
FROM salaries;
```

Value Functions

- `LAG(col, n)`: Wert der n-ten Zeile *davor*.
- `LEAD(col, n)`: Wert der n-ten Zeile *danach*.

-- Vergleich mit dem Gehalt des Vorgängers (zeitlich)

SELECT

emp_no,

salary,

LAG(salary) OVER (PARTITION BY emp_no ORDER BY from_date) as prev_salary

FROM salaries;

Rating

Vielen Dank für die Teilnahme am Seminar!

Bitte nehmt euch ein wenig Zeit und lasst ein Feedback da. Dies hilft uns sehr, die Qualität unserer Schulungen zu verbessern.

Hier der Link:

👉 [Click](#)

